



Red Hat OpenShift AI Self-Managed 2.25

OpenShift AI tutorial - Fraud detection example

Use OpenShift AI to train and deploy an example fraud detection model

Red Hat OpenShift AI Self-Managed 2.25 OpenShift AI tutorial - Fraud detection example

Use OpenShift AI to train and deploy an example fraud detection model

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Step-by-step guidance for using OpenShift AI to train an example model in JupyterLab, deploy the model, refine the model by using automated pipelines, and train the model by using distributed computing frameworks.

Table of Contents

CHAPTER 1. INTRODUCTION	3
1.1. ABOUT THE EXAMPLE FRAUD DETECTION MODEL	3
1.2. BEFORE YOU BEGIN	3
CHAPTER 2. SETTING UP A PROJECT AND STORAGE	5
2.1. NAVIGATING TO THE OPENSIFT AI DASHBOARD	5
2.2. SETTING UP YOUR DATA SCIENCE PROJECT	6
2.3. STORING DATA WITH CONNECTIONS	8
2.3.1. Running a script to install local object storage buckets and create connections	8
2.3.2. Creating connections to your own S3-compatible object storage	12
2.4. ENABLING DATA SCIENCE PIPELINES	16
2.5. SETTING UP KUEUE RESOURCES	19
CHAPTER 3. CREATING A WORKBENCH AND USING NOTEBOOKS	22
3.1. CREATING A WORKBENCH AND SELECTING A WORKBENCH IMAGE	22
3.2. IMPORTING THE TUTORIAL FILES INTO THE JUPYTERLAB ENVIRONMENT	25
3.3. RUNNING CODE IN A NOTEBOOK	28
3.4. TRAINING A MODEL	30
CHAPTER 4. DEPLOYING AND TESTING A MODEL	32
4.1. PREPARING A MODEL FOR DEPLOYMENT	32
4.2. DEPLOYING A MODEL	32
4.2.1. Deploying a model on a single-model server	33
4.2.2. Deploying a model on a multi-model server	36
4.3. TESTING THE MODEL API	39
CHAPTER 5. IMPLEMENTING PIPELINES	40
5.1. AUTOMATING WORKFLOWS WITH DATA SCIENCE PIPELINES	40
5.1.1. Creating a pipeline	40
5.1.2. Adding nodes to your pipeline	43
5.1.3. Specifying the training file as a dependency	44
5.1.4. Creating and storing the ONNX-formatted output file	45
5.1.5. Configuring the connection to storage	46
5.1.6. Running your pipeline	49
5.2. RUNNING A DATA SCIENCE PIPELINE GENERATED FROM PYTHON CODE	52
CHAPTER 6. RUNNING A DISTRIBUTED WORKLOAD	56
6.1. DISTRIBUTING TRAINING JOBS	56
6.1.1. Distributing training jobs with Ray	56
6.1.2. Distributing training jobs with the Training Operator	57
CHAPTER 7. CONCLUSION	58

CHAPTER 1. INTRODUCTION

Welcome. In this tutorial, you learn how to use data science and artificial intelligence (AI) and machine learning (ML) in an OpenShift development workflow.

You complete the following tasks in [Red Hat OpenShift AI](#) without the need to install anything on your computer:

- Explore a pre-trained fraud detection model by using a Jupyter notebook.
- Deploy the model by using OpenShift AI model serving.
- Refine and train the model by using automated pipelines.
- Learn how to train the model by using distributed computing frameworks.

1.1. ABOUT THE EXAMPLE FRAUD DETECTION MODEL

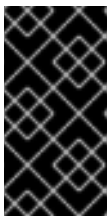
The example fraud detection model monitors credit card transactions for potential fraudulent activity. It analyzes the following credit card transaction details:

- The geographical distance from an earlier credit card transaction.
- The price of the current transaction, compared to the median price of all the user's transactions.
- Whether the user completed the transaction by using the hardware chip in the credit card, by entering a PIN number, or by making an online purchase.

Based on this data, the model outputs the likelihood of the transaction being fraudulent.

1.2. BEFORE YOU BEGIN

- You must have access to an OpenShift cluster which has Red Hat OpenShift AI installed. If you do not have access to a cluster that has an instance of OpenShift AI, see the [Red Hat OpenShift AI page on the Red Hat Developer website](#). There, you can create an account and access the [free Red Hat Developer Sandbox](#) or you can learn how to install OpenShift AI on [your own OpenShift cluster](#).



IMPORTANT

If your cluster uses self-signed certificates, before you begin the tutorial, your OpenShift AI administrator must add self-signed certificates for OpenShift AI as described in [Working with certificates](#) (Self-Managed) or [Working with certificates](#) (Cloud Service).

- Make sure that the version of this tutorial matches the OpenShift AI version on your cluster. Red Hat OpenShift AI v2.25 is the version described in this tutorial.
 1. To find the OpenShift AI version for your cluster:
 - a. Log in to the OpenShift AI dashboard.
 - b. In the top navigation bar, click the help icon () and then select **About**. The **About** page shows the installed version.

2. Check the [Red Hat OpenShift AI Supported Configurations](#) page to verify that your installed version is a supported version.
3. If this version of the tutorial does not match the installed version, access the matching version of the tutorial by navigating to the [top-level Red Hat OpenShift AI documentation page](#) and selecting the matching version from the drop-down list.

If you're ready, [start the tutorial](#).

CHAPTER 2. SETTING UP A PROJECT AND STORAGE

2.1. NAVIGATING TO THE OPENSIFT AI DASHBOARD

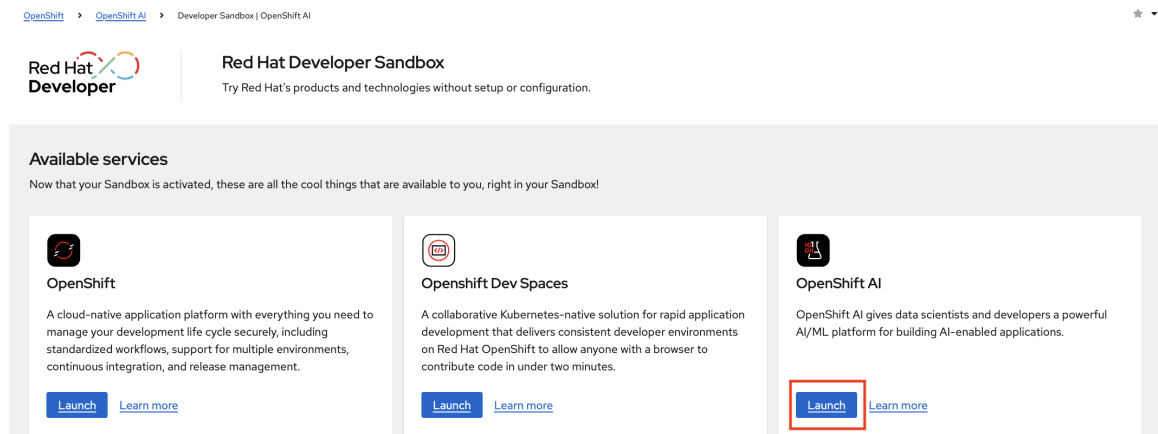
You can access the OpenShift AI dashboard from your OpenShift console.

Procedure

- How you open the OpenShift AI dashboard depends on your OpenShift environment:

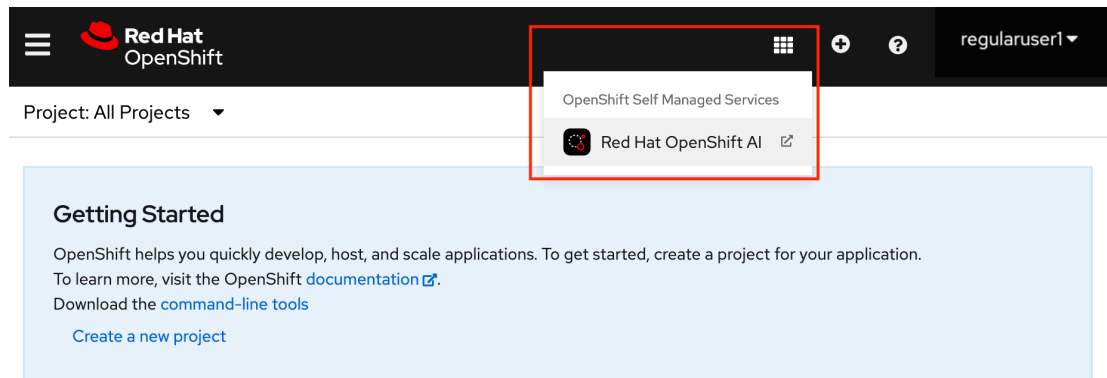
- If you are using the Red Hat Developer Sandbox**

After you log in to the Sandbox, in the Red Hat OpenShift AI card, click **Launch**.

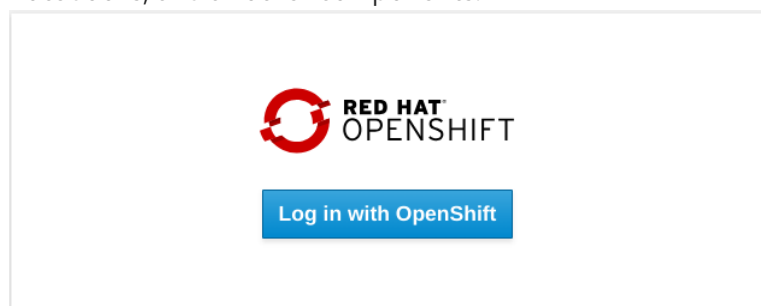


- If you are using your own OpenShift cluster:**

- After you log in to the OpenShift console, click the application launcher icon on the header.



- When prompted, log in to the OpenShift AI dashboard by using your OpenShift credentials. OpenShift AI uses the same credentials as OpenShift for the dashboard, notebooks, and all other components.

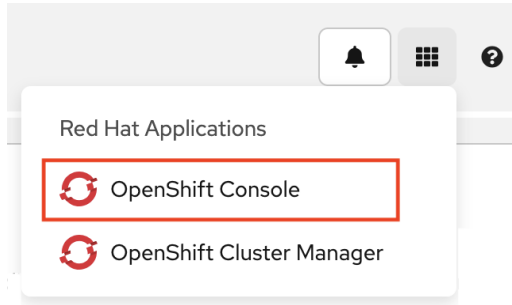


The OpenShift AI dashboard shows the **Home** page.



NOTE

You can navigate back to the OpenShift console by clicking the application launcher to access the OpenShift console.



For now, stay in the OpenShift AI dashboard.

Next step

[Setting up your data science project](#)

2.2. SETTING UP YOUR DATA SCIENCE PROJECT

To implement a data science workflow, you must create a data science project (as described in the following procedure). Projects help your team to organize and work together on resources within separated namespaces. From a project you can create many workbenches, each with their own IDE environment (for example, JupyterLab), and each with their own connections and cluster storage. In addition, the workbenches can share models and data with pipelines and model servers.

Prerequisites

- You have logged in to **Red Hat OpenShift AI**.

Procedure

1. On the navigation menu, select **Data science projects**. This page lists any existing projects that you have access to.
2. **If you are using the Red Hat Developer Sandbox**, it provides a default data science project (for example, **myname-dev**). Select it and skip to the **Verification** section.
If you are using your own OpenShift cluster, you can select an existing project (if any) or create a new one. Click **Create project**.



NOTE

You can start a Jupyter notebook by clicking the **Start basic workbench** button, selecting a notebook image, and clicking **Start server**. However, in that case, it is a one-off Jupyter notebook run in isolation.

3. In the **Create project** modal, enter a display name and description.

4. Click **Create**.

Verification

You can see your project's initial state. Individual tabs show more information about the project components and project access permissions:

[Data Science Projects](#) > Fraud detection


Fraud detection


My Fraud Detection project

Overview Workbenches Pipelines Models Cluster storage Connections Permissions

- **Workbenches** are instances of your development and experimentation environment. They typically contain individual development environments (IDEs), such as JupyterLab, RStudio, and Visual Studio Code.
- **Pipelines** contain the data science pipelines which run within the project.
- **Models** for quickly serving a trained model for real-time inference. You can have many model servers per data science project. One model server can host many models.
- **Cluster storage** is a persistent volume that retains the files and data you're working on within a workbench. A workbench has access to one or more cluster storage instances.
- **Connections** contain required configuration parameters for connecting to a data source, such as an S3 object bucket.
- **Permissions** define which users and groups can access the project.

Next step

[Storing data with connections](#)

2.3. STORING DATA WITH CONNECTIONS

Add connections to workbenches to connect your project to data inputs and object storage buckets. A connection is a resource that has the configuration parameters needed to connect to a data source or data sink, such as an AWS S3 object storage bucket.

For this tutorial, you run a provided script that creates the following local Minio storage buckets for you:

- **My Storage** - Use this bucket for storing your models and data. You can reuse this bucket and its connection for your notebooks and model servers.
- **Pipelines Artifacts** - Use this bucket as storage for your pipeline artifacts. When you create a pipeline server, you need a pipeline artifacts bucket. For this tutorial, create this bucket to separate it from the first storage bucket for clarity.



NOTE

Although you can use one storage bucket for both storing models and data and for storing pipeline artifacts, this tutorial follows best practice and uses separate storage buckets for each purpose.

The provided script also creates a connection to each storage bucket.

To run the script that installs local MinIO storage buckets and creates connections to them, follow the steps in [Running a script to install local object storage buckets and create connections](#).



NOTE

If you want to use your own S3-compatible object storage buckets (instead of using the provided script), follow the steps in [Creating connections to your own S3-compatible object storage](#).

2.3.1. Running a script to install local object storage buckets and create connections

For convenience, run a script (provided in the following procedure) that automatically completes these tasks:

- Creates a Minio instance in your project.
- Creates two storage buckets in that Minio instance.
- Generates a random user id and password for your Minio instance.
- Creates two connections in your project, one for each bucket and both using the same credentials.
- Installs required network policies for service mesh functionality.

The [guide for deploying Minio](#) is the basis for this script.



IMPORTANT

The Minio-based Object Storage that the script creates is **not** meant for production usage.



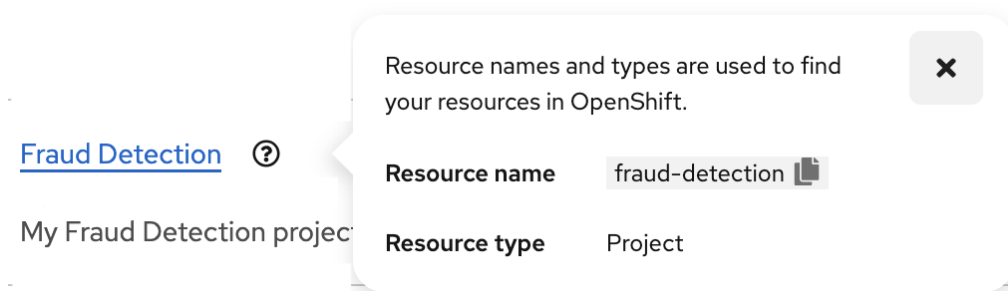
NOTE

If you want to connect to your own storage, see [Creating connections to your own S3-compatible object storage](#).

Prerequisites

You must know the OpenShift resource name for your data science project so that you run the provided script in the correct project. To get the project's resource name:

In the OpenShift AI dashboard, select **Data science projects** and then click the ? icon next to the project name. A text box opens with information about the project, including its resource name:



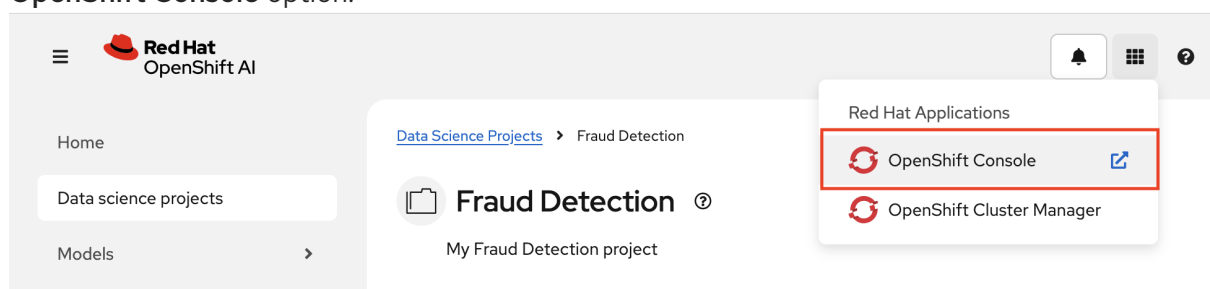
NOTE

The following procedure describes how to run the script from the OpenShift console. If you are knowledgeable in OpenShift and can access the cluster from the command line, instead of following the steps in this procedure, you can use the following command to run the script:

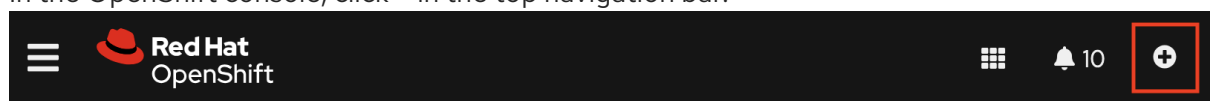
```
oc apply -n <your-project-name/> -f https://github.com/rh-aieservices-bu/fraud-detection/raw/main/setup/setup-s3.yaml
```

Procedure

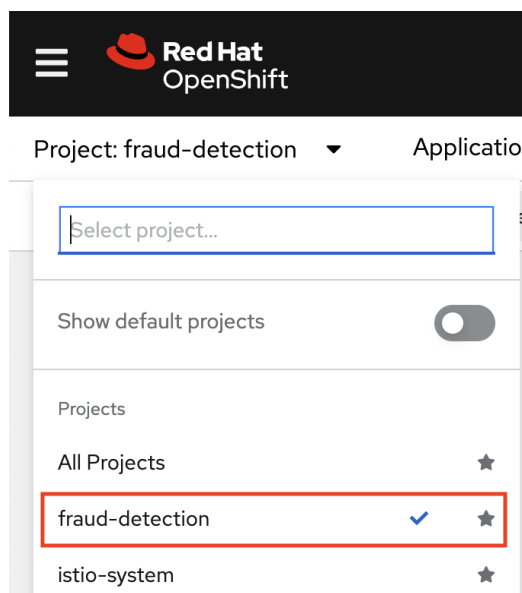
1. In the OpenShift AI dashboard, click the application launcher icon and then select the **OpenShift Console** option.



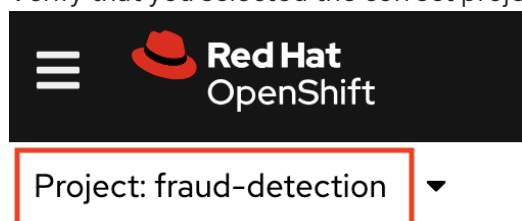
2. In the OpenShift console, click + in the top navigation bar.



3. Select your project from the list of projects.



4. Verify that you selected the correct project.



Import YAML

Drag and drop YAML or JSON files into

5. Copy the following code and paste it into the **Import YAML** editor.



NOTE

This code gets and applies the **setup-s3-no-sa.yaml** file.

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: demo-setup
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: demo-setup-edit
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: edit
subjects:
  - kind: ServiceAccount
```

```

    name: demo-setup
---
apiVersion: batch/v1
kind: Job
metadata:
  name: create-s3-storage
spec:
  selector: {}
  template:
    spec:
      containers:
      - args:
        - -ec
        - |-
          echo -n 'Setting up Minio instance and connections'
          oc apply -f https://github.com/rh-aisservices-bu/fraud-detection/raw/main/setup/setup-
s3-no-sa.yaml
        command:
        - /bin/bash
        image: image-registry.openshift-image-registry.svc:5000/openshift/tools:latest
        imagePullPolicy: IfNotPresent
        name: create-s3-storage
      restartPolicy: Never
      serviceAccount: demo-setup
      serviceAccountName: demo-setup

```

6. Click **Create**.

Verification

1. In the OpenShift console, there is a "Resources successfully created" message and a list of the following resources:
 - **demo-setup**
 - **demo-setup-edit**
 - **create-s3-storage**
2. In the OpenShift AI dashboard:
 - a. Select **Data science projects** and then click the name of your project, **Fraud detection**.
 - b. Click **Connections**. There are two connections listed: **My Storage** and **Pipeline Artifacts**.

[Data Science Projects](#) > Fraud Detection **Fraud Detection**

My Fraud Detection project

Actions 

Overview

Workbenches

Pipelines

Models

Cluster storage

Connections

Permissions

Settings

 Connections

Add connection

Name 	Type 	Model serving compatibility	Connected resources
My Storage 	S3 compatible object storage - v1	S3 compatible object storage	- 
Pipeline Artifacts 	S3 compatible object storage - v1	S3 compatible object storage	- 

**IMPORTANT**

If your cluster uses self-signed certificates, your OpenShift AI administrator might need to configure a certificate authority (CA) to securely connect to the S3 object storage, as described in [Accessing S3-compatible object storage with self-signed certificates](#) (Self-Managed) or [Accessing S3-compatible object storage with self-signed certificates](#) (Cloud Service).

Next step

If you want to complete the pipelines section of this tutorial, go to [Enabling data science pipelines](#).

Otherwise, skip to [Creating a workbench](#).

2.3.2. Creating connections to your own S3-compatible object storage

If you have existing S3-compatible storage buckets that you want to use for this tutorial, you must create a connection to one storage bucket for saving your data and models. If you want to complete the pipelines section of this tutorial, create another connection to a different storage bucket for saving pipeline artifacts.

**NOTE**

If you do not have your own s3-compatible storage, or if you want to use a disposable local Minio instance instead, skip this task and follow the steps in [Running a script to install local object storage buckets and create connections](#). The provided script automatically completes the following tasks for you: creates a Minio instance in your project, creates two storage buckets in that Minio instance, creates two connections in your project, one for each bucket and both using the same credentials, and installs required network policies for service mesh functionality.

Prerequisites

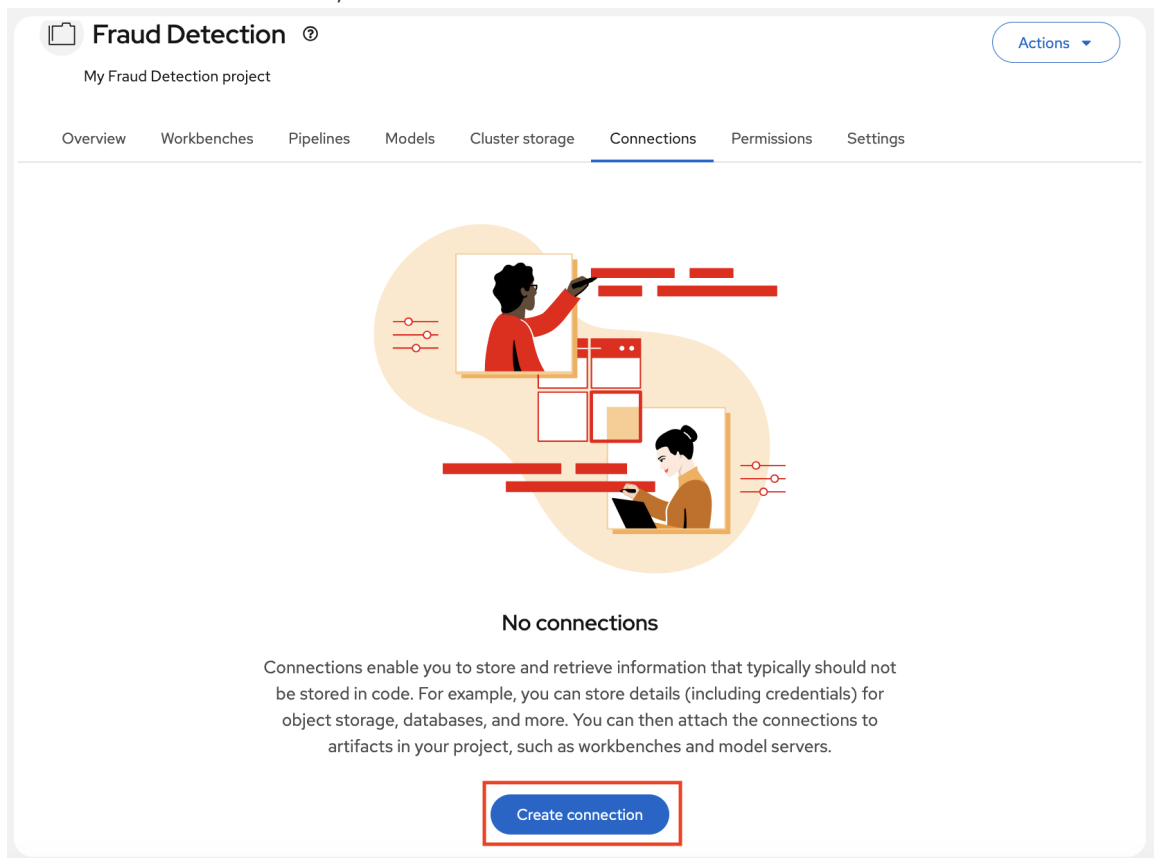
To create connections to your existing S3-compatible storage buckets, you need the following credential information for the storage buckets:

- Endpoint URL
- Access key
- Secret key
- Region
- Bucket name

If you do not have this information, contact your storage administrator.

Procedure

1. Create a connection for saving your data and models:
 - a. In the OpenShift AI dashboard, navigate to the page for your data science project.
 - b. Click the **Connections** tab, and then click **Create connection**.



- c. In the **Add connection** modal, for the **Connection type** select **S3 compatible object storage - v1**.
- d. Complete the **Add connection** form and name your connection **My Storage**. This connection is for saving your personal work, including data and models.

Add connection ✕

Connection type *

S3 compatible object storage - v1 ▼

Connect to storage systems that are compatible with Amazon S3, enabling integration with other S3-compatible services and applications.
[View connection type details](#)

Connection details

Connection name *

My Storage

The resource name will be **my-storage**.
[Edit resource name](#) ?

Connection description

Access key * ?

kadhaksjdhsuidacn

Secret key * ?

..... 🗑

Endpoint * ?

https://minio-s3-fraud-detection-example.com

Region ?

us-east

Create **Cancel**

e. Click **Create**.

2. Create a connection for saving pipeline artifacts:



NOTE

If you do not intend to complete the pipelines section of the tutorial, you can skip this step.

a. Click **Add connection**.

b. Complete the form and name your connection **Pipeline Artifacts**.

Add connection ✕

Connection type *

S3 compatible object storage - v1 ▼

Connect to storage systems that are compatible with Amazon S3, enabling integration with other S3-compatible services and applications.
[View connection type details](#)

Connection details

Connection name *

Pipeline Artifacts

The resource name will be **pipeline-artifacts**.

[Edit resource name](#) ?

Connection description

Access key * ?

yuieayoaifiasbyr5jkskdhka

Secret key * ?

..... 🗑

Endpoint * ?

https://minio-s3-fraud-detection-example.com

Region ?

us-east

Create Cancel

- c. Click **Create**.

Verification

In the **Connections** tab for the project, check to see that your connections are listed.

 Fraud Detection

My Fraud Detection project

Actions 

- Overview
- Workbenches
- Pipelines
- Models
- Cluster storage
- Connections
- Permissions
- Settings

 Connections

Add connection

Name 	Type 	Model serving compatibility	Connected resources
My Storage 	S3 compatible object storage - v1	S3 compatible object storage	- 
Pipeline Artifacts 	S3 compatible object storage - v1	S3 compatible object storage	- 



IMPORTANT

If your cluster uses self-signed certificates, your OpenShift AI administrator might need to provide a certificate authority (CA) to securely connect to the S3 object storage, as described in [Accessing S3-compatible object storage with self-signed certificates](#) (Self-Managed) or [Accessing S3-compatible object storage with self-signed certificates](#) (Cloud Service).

Next step

If you want to complete the pipelines section of this tutorial, go to [Enabling data science pipelines](#).
Otherwise, skip to [Creating a workbench](#).

2.4. ENABLING DATA SCIENCE PIPELINES

You must prepare your tutorial environment so that you can use data science pipelines.



NOTE

If you do not intend to complete the pipelines section of this tutorial you can skip this step and move on to the next section, [Setting up Kueue resources](#).

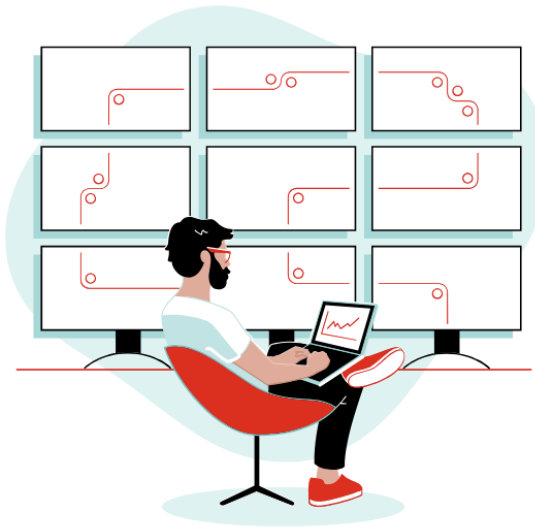
Later in this tutorial, you implement an example pipeline by using the JupyterLab Elyra extension. With Elyra, you can create a visual end-to-end pipeline workflow that executes in OpenShift AI.

Prerequisites

- You have installed local object storage buckets and created connections, as described in [Storing data with connections](#).

Procedure

1. In the OpenShift AI dashboard, on the **Fraud Detection** page, click the **Pipelines** tab.
2. Click **Configure pipeline server**.



Configure a pipeline server

To create or use pipelines, you must first configure a pipeline server in this project. A pipeline server provides the infrastructure necessary for the pipeline to execute steps, track results, and manage runs.

[Configure pipeline server](#)

3. In the **Configure pipeline server** form, in the **Access key** field next to the key icon, click the dropdown menu and then click **Pipeline Artifacts**.
The **Configure pipeline server** form autofills with credentials for the connection.

Configure pipeline server

Configuring a pipeline server enables you to create and manage pipelines.

! Pipeline server configuration cannot be edited after creation. To use a different configuration after creation, delete the pipeline server and create a new one.

Object storage connection

To store pipeline artifacts. Must be S3 compatible

Autofill from connection

My Storage
.....

Pipeline Artifacts
.....

Access key *

Secret key *

Endpoint *

Region *

Configure pipeline server Cancel

- In the **Advanced Settings** section, leave the default values.
- Click **Configure pipeline server**.
- Wait until the loading spinner disappears and **Start by importing a pipeline** is displayed.



IMPORTANT

You must wait until the pipeline configuration is complete before you continue and create your workbench. If you create your workbench before the pipeline server is ready, your workbench cannot submit pipelines to it.

If you have waited more than 5 minutes, and the pipeline server configuration does not complete, you can delete the pipeline server and create it again.

Import pipeline

View pipeline server configuration

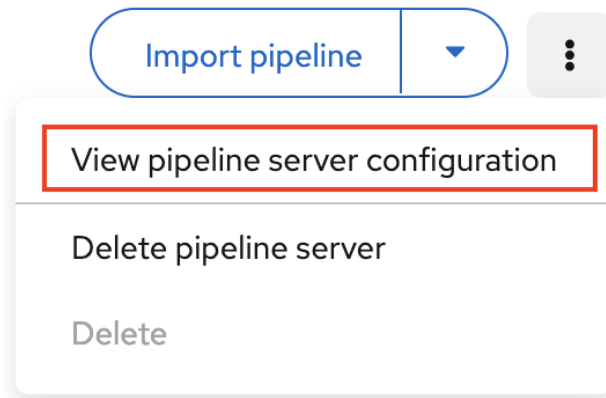
Delete pipeline server

Delete

You can also ask your OpenShift AI administrator to verify that they applied self-signed certificates on your cluster as described in [Working with certificates](#) (Self-Managed) or [Working with certificates](#) (Cloud Service).

Verification

1. Navigate to the **Pipelines** tab for the project.
2. Next to **Import pipeline**, click the action menu (⋮) and then select **View pipeline server configuration**.



An information box opens and displays the object storage connection information for the pipeline server.

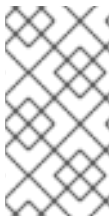
Next step

[Setting up Kueue resources](#)

2.5. SETTING UP KUEUE RESOURCES

You must prepare your tutorial environment so that you can use Kueue for distributing training with the Training Operator.

In the *Distributing training jobs with the Training Operator* section of this tutorial, you implement a distributed training job by using Kueue for managing job resources. With Kueue, you can manage cluster resource quotas and how different workloads consume them.

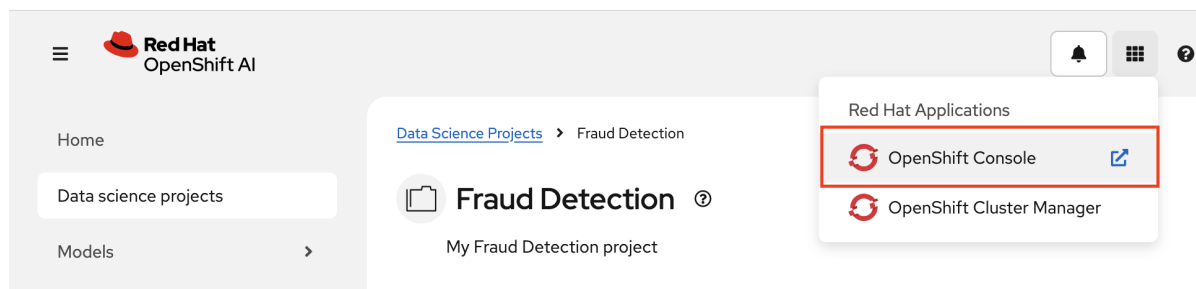


NOTE

If you are using the Red Hat Developer Sandbox, or if you do not intend to use Kueue to schedule your training jobs in the *Distributing training jobs with the Training Operator* section of this tutorial, skip this procedure and continue to the next section, [Creating a workbench and selecting a workbench image](#).

Procedure

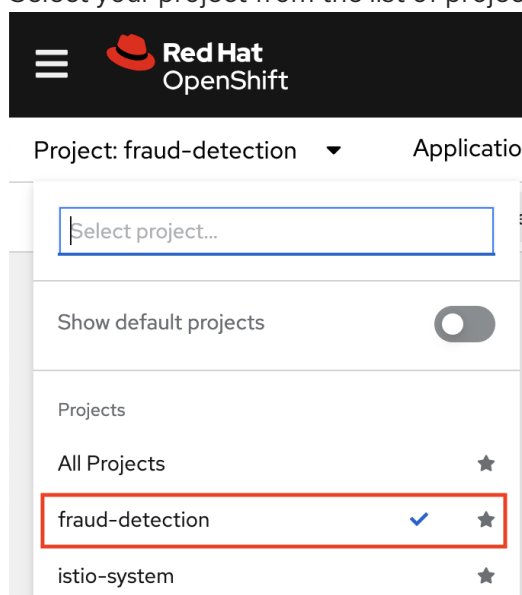
1. In the OpenShift AI dashboard, click the application launcher icon and then select the **OpenShift Console** option.



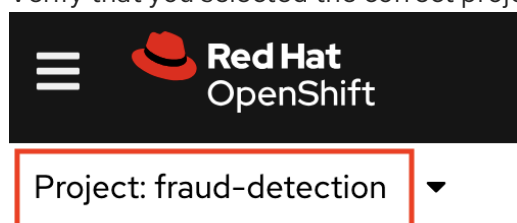
- In the OpenShift console, click + in the top navigation bar.



- Select your project from the list of projects.



- Verify that you selected the correct project.



Import YAML

Drag and drop YAML or JSON files into

- Copy the following code and paste it into the **Import YAML** editor.

```
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: ResourceFlavor
metadata:
  name: default-flavor
---
```



```

apiVersion: kueue.x-k8s.io/v1beta1
kind: ClusterQueue
metadata:
  name: cluster-queue
spec:
  namespaceSelector: {}
  resourceGroups:
  - coveredResources: ["cpu", "memory"]
    flavors:
    - name: "default-flavor"
      resources:
      - name: "cpu"
        nominalQuota: 4
      - name: "memory"
        nominalQuota: 8Gi
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  name: local-queue
spec:
  clusterQueue: cluster-queue

```

6. Click **Create**.

Verification

1. In the OpenShift console, there is a "Resources successfully created" message with a list of the following resources:
 - **default-flavor**
 - **cluster-queue**
 - **local-queue**

Next step

[Creating a workbench and selecting a notebook image](#)

CHAPTER 3. CREATING A WORKBENCH AND USING NOTEBOOKS

3.1. CREATING A WORKBENCH AND SELECTING A WORKBENCH IMAGE

A workbench is an instance of your development and experimentation environment. When you create a workbench, you select a workbench image that has the tools and libraries that you need for developing models.

Prerequisites

- You created a **My Storage** connection as described in [Storing data with connections](#).
- If you intend to complete the pipelines section of this tutorial, you configured a pipeline server as described in [Enabling data science pipelines](#).

Procedure

1. Navigate to the project detail page for the data science project that you created in [Setting up your data science project](#).
2. Click the **Workbenches** tab, and then click the **Create workbench** button.



Start by creating a workbench

A workbench is an isolated area where you can work with models in your preferred IDE, such as a Jupyter notebook. You can add accelerators and data connections, create pipelines, and add cluster storage in your workbench.

Create workbench

3. Fill out the name and description.

Name *

Fraud Detection

The resource name will be **fraud-detection**.[Edit resource name](#) **Description**

My Fraud Detection workbench

Red Hat provides several supported workbench images. In the **Workbench image** section, you can select one of the default images or a custom image that an administrator has set up for you. The **Tensorflow** image has the libraries needed for this tutorial.

4. Select the latest **Tensorflow** image.

Workbench image**Image selection ***

Jupyter | TensorFlow | CUDA | Python 3.12

CUDA v12.6, Python v3.12, TensorFlow v2.18

 [View package information](#)

5. Choose a small deployment.

Deployment size**Container size**

Small

Limits: 2 CPU, 8GiB Memory Requests: 1 CPU, 8GiB Memory

6. If your OpenShift cluster has available GPUs, the **Create workbench** form includes an **Accelerator** option. Select **None**. This tutorial does not require any GPUs.
7. Leave the default environment variables and storage options.

Environment variables

[+ Add variable](#)

Cluster storage

[Attach existing storage](#)

[Create storage](#)

Name	Storage size	Mount path	
fraud-detection-storage	Max 20GiB	/opt/app-root/src/	

- 8. For **Connections**, click **Attach existing connection**.
- 9. Select **My Storage** (the object storage that you configured earlier) and then click **Attach**.

Connections

[Attach existing connections](#)

[Create connection](#)

Name	Type	
My Storage	S3 compatible object storage - v1	

- 10. Click **Create workbench**.

Verification

In the **Workbenches** tab for the project, the status of the workbench changes from **Starting** to **Running**.

[Data Science Projects](#) > [Fraud Detection](#)

Fraud Detection

[Actions](#)

Overview

Workbenches

Pipelines

Models

Cluster storage

Connections

Permissions

Settings

Workbenches

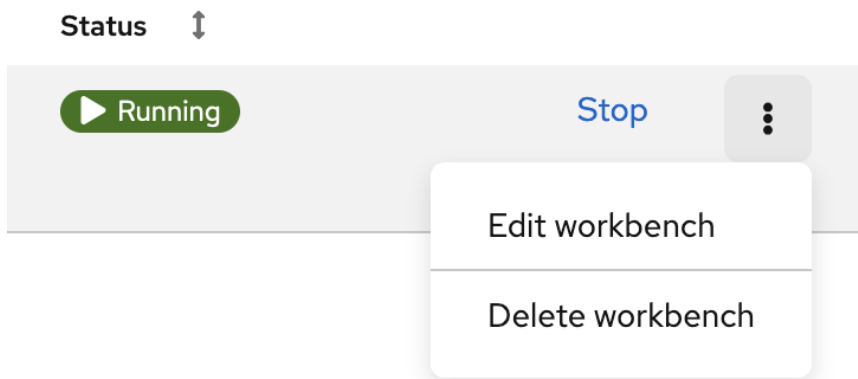
Create workbench

Name	Workbench image	Container size	Status	
Fraud Detection My Fraud Detection workbench	TensorFlow Latest CUDA v12.6, Python v3.11, TensorFlow v2.18	Small	Running	Stop



NOTE

If you made a mistake, you can edit the workbench to make changes.



Next step

[Importing the tutorial files into the JupyterLab environment](#)

3.2. IMPORTING THE TUTORIAL FILES INTO THE JUPYTERLAB ENVIRONMENT

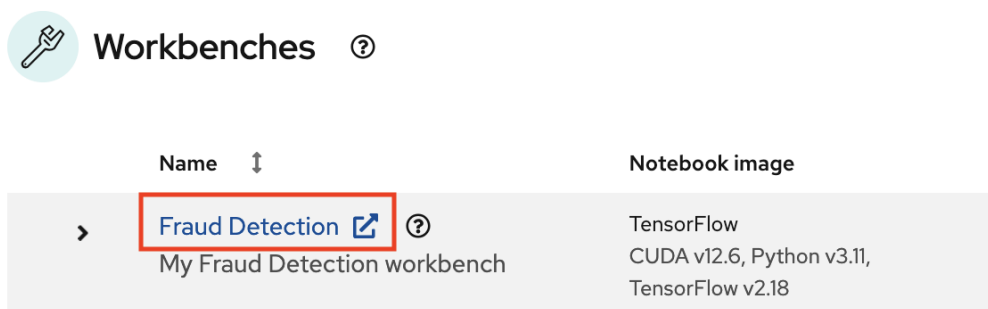
The JupyterLab environment is a web-based environment, but everything you do inside it happens on **Red Hat OpenShift AI** and is powered by the **OpenShift** cluster. This means that, without having to install and maintain anything on your own computer, and without using valuable local resources such as CPU, GPU and RAM, you can conduct your data science work in this powerful and stable managed environment.

Prerequisites

You created a workbench, as described in [Creating a workbench and selecting a workbench image](#).

Procedure

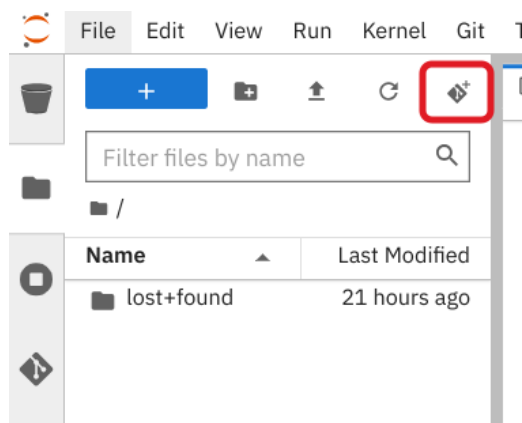
1. Click the link for your workbench. If prompted, log in and allow JupyterLab to authorize your user.



Your JupyterLab environment window opens.

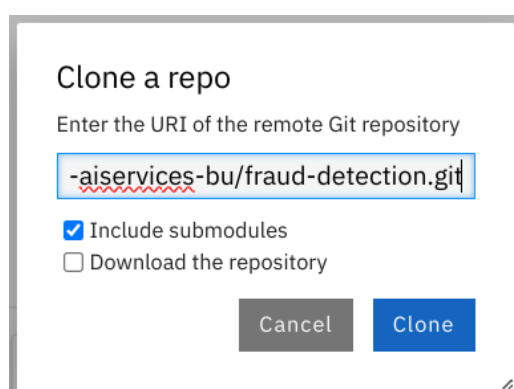
This file-browser window shows the files and folders that are saved inside your own personal space in OpenShift AI.

2. Bring the content of this tutorial inside your JupyterLab environment:
 - a. On the toolbar, click the **Git Clone** icon:

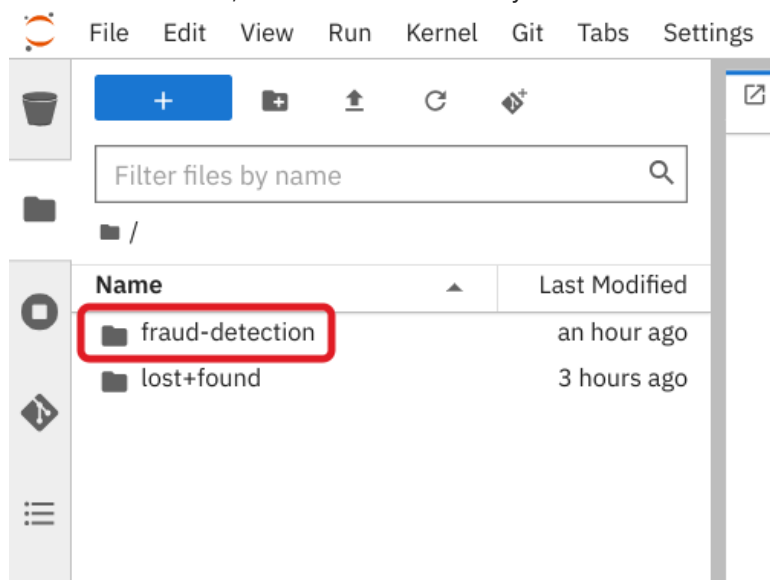


- b. Enter the following tutorial Git **https** URL:

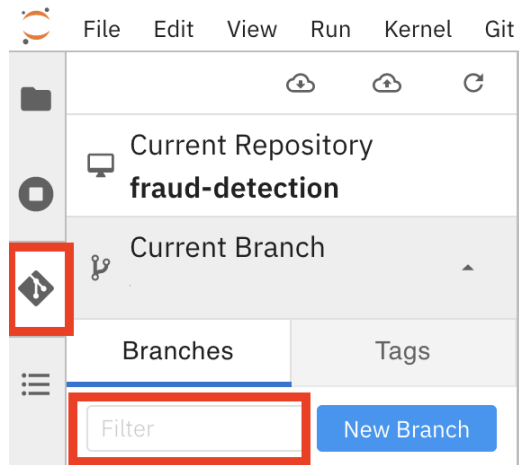
```
https://github.com/rh-aishervices-bu/fraud-detection.git
```



- c. Select the **Include submodules** option, and then click **Clone**.
- d. In the file browser, double-click the newly-created **fraud-detection** folder.



- e. In the left navigation bar, click the **Git** icon, and then click **Current Branch** to expand the branches and tags selector panel.
- i. On the **Branches** tab, in the **Filter** field, enter **v2.25**.



- f. Select **origin/v2.25**.
The current branch changes to **v2.25**.

Verification

In the file browser, view the notebooks that you cloned from Git.

/ fraud-detection /	
Name	
assets	
data	
kfto-scripts	
pipeline	
ray-scripts	
setup	
utils	
workshop	
0_sandbox.ipynb	
1_experiment_train.ipynb	
2_save_model.ipynb	
3_rest_requests_multi_model.ipynb	
4_grpc_requests_multi_model.ipynb	
5_rest_requests_single_model.ipynb	
6 Train Save.pipeline	
7_get_data_train_upload.yaml	
8_distributed_training.ipynb	
• 9_distributed_training_kfto.ipynb	
LICENSE	
README.md	

Next step

[Running code in a notebook](#)

or

[Training a model](#)

3.3. RUNNING CODE IN A NOTEBOOK

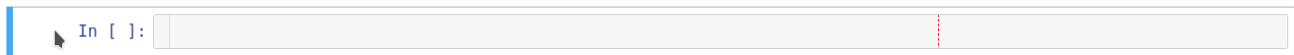


NOTE

If you're already at ease with Jupyter, you can [skip to the next section](#).

A notebook is an environment where you have *cells* that can display formatted text or code.

This is an empty cell:



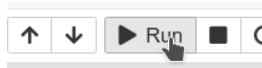
This is a cell with some code:



Code cells contain Python code that you can run interactively. You can edit the code and then run it. The code does not run on your computer or in the browser, but directly in your connected environment, **Red Hat OpenShift AI** in your case.

You can run a code cell from the notebook interface or from the keyboard:

- **From the user interface:** Select the cell (by clicking inside the cell or to the left side of the cell) and then click **Run** from the toolbar.



- **From the keyboard:** Press **CTRL + ENTER** to run a cell or press **SHIFT + ENTER** to run the cell and automatically select the next one.

After you run a cell, you can see the result of its code and information about when the code in the cell ran, as shown in this example:



When you save a notebook, the code and the results are saved. You can reopen the notebook to view the results without having to run the program again, and still having access to the code.

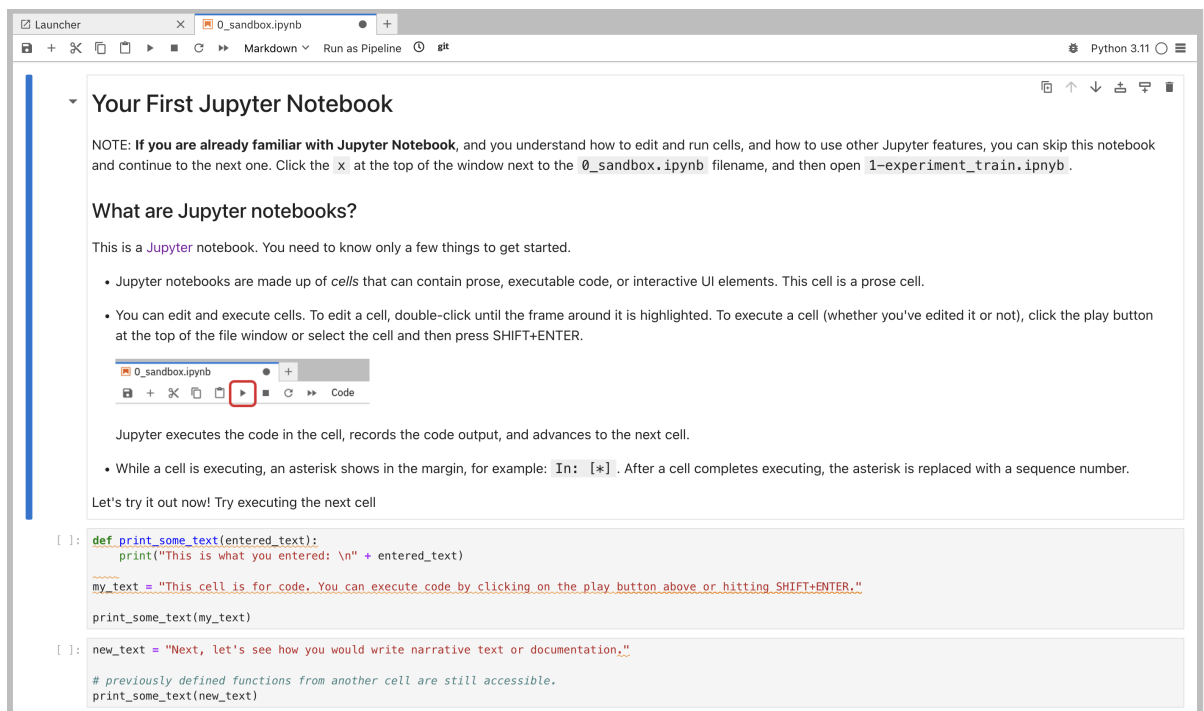
Notebooks are so named because they are like a physical *notebook*: you can take notes about your experiments, along with the code itself, including any parameters that you set. You can see the output of the experiment inline (this is the result after a cell runs), along with all the notes that you want to take. To take notes, from the menu switch the cell type from **Code** to **Markdown**.

Prerequisites

- You have imported the tutorial files into your JupyterLab environment as described in [Importing the tutorial files into the JupyterLab environment](#).

Procedure

1. In your JupyterLab environment, locate the **0_sandbox.ipynb** file and double-click it to launch the notebook. The notebook opens in a new tab in the content section of the environment.



- Experiment by, for example, running the existing cells, adding more cells and creating functions. You can do what you want – it is your environment and there is no risk of breaking anything or impacting other users. This environment isolation is also a great advantage brought by OpenShift AI.
- Optionally, create a new notebook in which the code cells are run by using a Python 3 kernel:
 - Create a new notebook by either selecting **File → New → Notebook** or by clicking the Python 3 tile in the Notebook section of the launcher window:



You can use different kernels, with different languages or versions, to run in your notebook.

Additional resources

- [the Jupyter site](#)

Next step

[Training a model](#)

3.4. TRAINING A MODEL

In your notebook environment, open the `1_experiment_train.ipynb` file and follow the instructions directly in the notebook. The instructions guide you through some simple data exploration, experimentation, and model training tasks.

The screenshot shows the JupyterLab interface. On the left is a file explorer for the 'fraud-detection' project, listing files like 'assets', 'data', 'pipeline', 'ray-scripts', 'setup', 'utils', 'workshop', '0_sandbox.ipynb', '1_experiment_train.ipynb', '2_save_model.ipynb', '3_rest_requests...', '4_xrpc_requests...', '5_rest_requests...', '6 Train Save.pipe...', '7_get_data_train...', '8 distributed_train...', 'LICENSE', and 'README.md'. The main area displays a notebook titled 'Experiment' with the following content:

Experiment

Install Python dependencies

```
[ ]: !pip install onnx onnxruntime tf2onnx
```

Import the dependencies for the model training code:

```
[ ]: import numpy as np
import pandas as pd
import datetime
from keras.models import Sequential
from keras.layers import Dense, Dropout, BatchNormalization, Activation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils import class_weight
import onnx
import pickle
from pathlib import Path
```

The output might show TensorFlow messages, such as a "Could not find TensorRT" warning. You can ignore these messages.

Load the CSV data

The CSV data that you use to train the model contains the following fields:

- **distancefromhome** - The distance from home where the transaction happened.
- **distancefromlast_transaction** - The distance from the last transaction that happened.
- **ratiotomedianpurchaseprice** - The ratio of purchased price compared to median purchase price.
- **repeat_retailer** - If it's from a retailer that already has been purchased from before.
- **used_chip** - If the credit card chip was used.
- **usedpinnumber** - If the PIN number was used.
- **online_order** - If it was an online order.

When you save the model, you convert the model to the portable Open Neural Network Exchange (ONNX) format. By using ONNX, you can transfer models between frameworks with minimal preparation and without the need for rewriting the models.

Next step

[Preparing a model for deployment](#)

CHAPTER 4. DEPLOYING AND TESTING A MODEL

4.1. PREPARING A MODEL FOR DEPLOYMENT

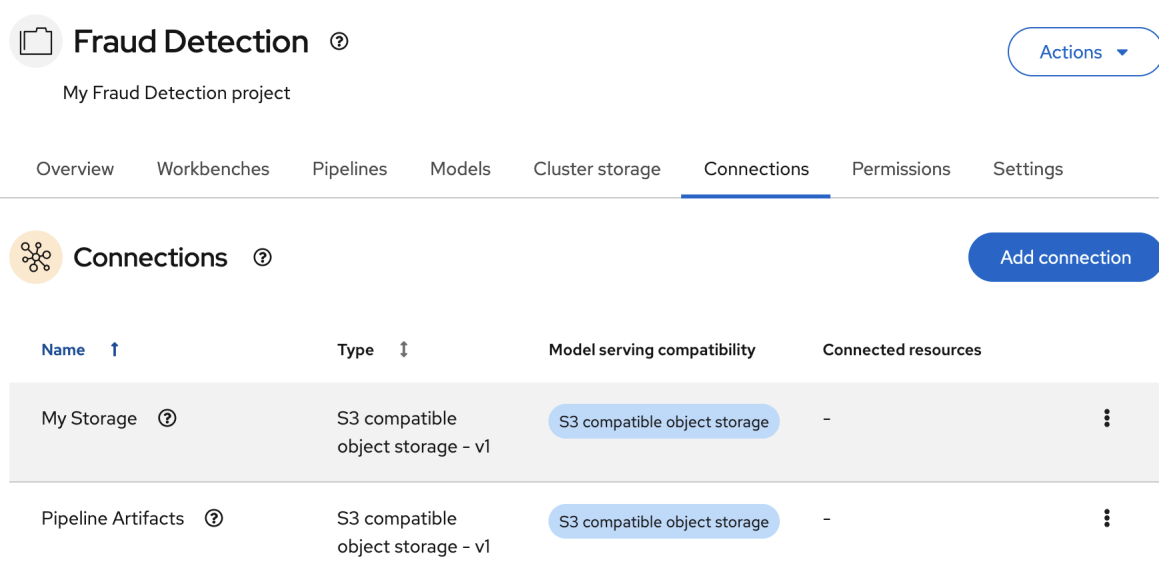
After you train a model, you can deploy it by using the OpenShift AI model serving capabilities.

To prepare a model for deployment, you must move the model from your workbench to your S3-compatible object storage. Use the connection that you created in the [Storing data with connections](#) section and upload the model from a notebook.

Prerequisites

- You created the **My Storage** connection and have added it to your workbench.

[Data Science Projects](#) > Fraud Detection



Fraud Detection ?

My Fraud Detection project

Overview Workbenches Pipelines Models Cluster storage **Connections** Permissions Settings

Connections ? Add connection

Name ↑	Type ↑	Model serving compatibility	Connected resources
My Storage ?	S3 compatible object storage - v1	S3 compatible object storage	-
Pipeline Artifacts ?	S3 compatible object storage - v1	S3 compatible object storage	-

Procedure

- In your JupyterLab environment, open the **2_save_model.ipynb** file.
- Follow the instructions in the notebook to make the model accessible in storage.

Verification

When you have completed the notebook instructions, the **models/fraud/1/model.onnx** file is in your object storage and it is ready for your model server to use.

Next step

[Deploying a model](#)

4.2. DEPLOYING A MODEL

Given that the model is accessible in storage and saved in the portable ONNX format, you can use an OpenShift AI model server to deploy it as an API.

OpenShift AI offers two options for model serving:

- **Single-model serving** – Each model in the project deploys on its own model server. This platform works well for large models or models that need dedicated resources.
- **Multi-model serving** – All models in the project deploy on the same model server. This platform is suitable for sharing resources among deployed models. Multi-model serving is the only option offered in the Red Hat Developer Sandbox environment.

For this tutorial, because you are deploying only one model, you can select either serving type. The steps for deploying the fraud detection model depend on the type of model serving platform that you select:

- [Deploying a model on a single-model server](#)
- [Deploying a model on a multi-model server](#)

4.2.1. Deploying a model on a single-model server

OpenShift AI single-model servers host only one model. You create a new model server and deploy your model to it.

Prerequisites

- A user with **admin** privileges has enabled the single-model serving platform on your OpenShift cluster.

Procedure

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Models** tab.

[Data Science Projects](#) > [Fraud Detection](#)


Fraud Detection ⓘ

My Fraud Detection project

Overview Workbenches Pipelines **Models** Cluster storage Connections Permissions Settings

Select the model serving type to be used when deploying from this project.

Single-model serving platform

Each model is deployed on its own model server. Choose this option when you want to deploy a large model such as a large language model (LLM).

Select single-model

Multi-model serving platform

Multiple models can be deployed on one shared model server. Choose this option when you want to deploy a number of small or medium-sized models that can share the server resources.

Select multi-model

i You can change the model serving type before the first model is deployed from this project. After deployment, switching types requires deleting all models and servers.



NOTE

Depending on the model serving configuration on your cluster, you might see only one model serving platform option.

2. In the **Single-model serving platform** tile, click **Select single-model**.
3. In the form, specify the following values:
 - a. For **Model deployment name**, type **fraud**.
 - b. For **Serving runtime**, select **OpenVINO Model Server**.
 - c. For **Model framework (name - version)**, select **onnx-1**.
 - d. For **Deployment mode**, select **Advanced**.
 - e. For **Existing connection**, select **My Storage**.
 - f. Type the path that leads to the version folder that has your model file: **models/fraud**
 - g. Leave the other fields with the default settings.

Deploy model

×

Configure properties for deploying your model

Project

Fraud Detection

Model deployment name *

Fraud

This is the name of the inference service created when the model is deployed.
The resource name will be **fraud**.

[Edit resource name](#) ⓘ

Serving runtime *

OpenVINO Model Server v2025.2 Global-scoped

You can optimize model performance by [configuring the parameters](#) of the selected serving runtime.

Model framework (name - version) *

onnx - 1

Deployment mode * ⓘ

Advanced

>
⚠ Token authentication service not installed
×

Number of model server replicas to deploy * ⓘ

Minimum replicas Maximum replicas

−

1

+

−

1

+

Model server size * ⓘ

Small

Limits: 2 CPU, 8GiB Memory Requests: 1 CPU, 4GiB Memory

Accelerator ⓘ

None

Model route

☐ Make deployed models available through an external route

Source model location

☒ Existing connection

Connection * ⓘ

My Storage

[View details](#)

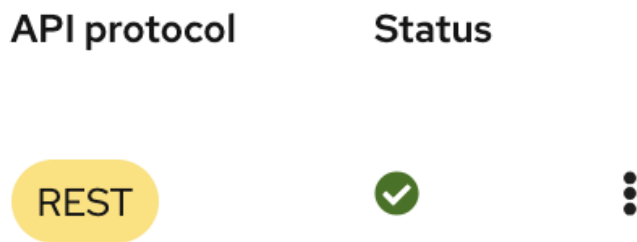
Deploy

Cancel

4. Click **Deploy**.

Verification

Notice the loading symbol under the **Status** section. The symbol changes to a checkmark when the deployment completes successfully.



Next step

Testing the model API

4.2.2. Deploying a model on a multi-model server

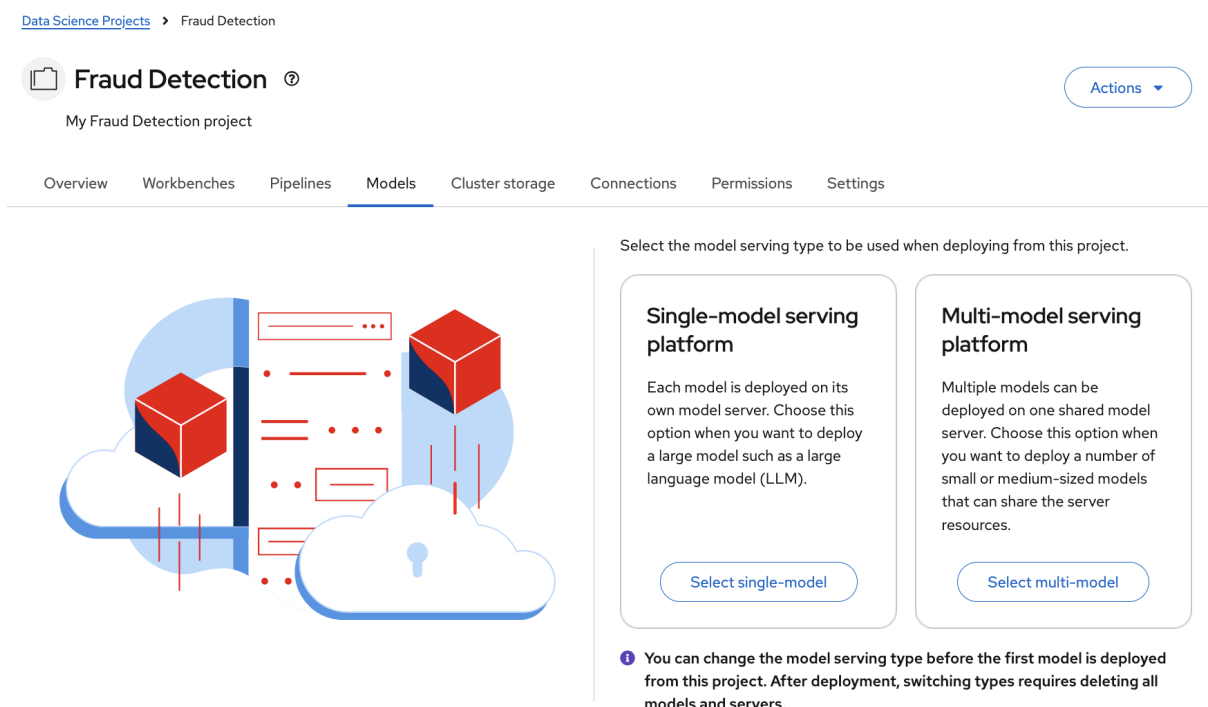
OpenShift AI multi-model servers can host several models at once. You create a new model server and deploy your model to it.

Prerequisites

- A user with **admin** privileges has enabled the multi-model serving platform on your OpenShift cluster.

Procedure

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Models** tab.



**NOTE**

Depending on how model serving has been configured on your cluster, you might see only one model serving platform option.

2. In the **Multi-model serving platform** tile, click **Select multi-model**.
3. In the form, specify the following values:
 - a. For **Model server name**, type a name, for example **Model Server**.
 - b. For **Serving runtime**, select **OpenVINO Model Server**.
 - c. Leave the other fields with the default settings.

Add model server

A model server specifies resources available for use by one or more supported models, and includes a serving runtime.

Model server name *

Model Server

The resource name will be **model-server**.

[Edit resource name](#) ?

Serving runtime *

OpenVINO Model Server

Model server replicas

Number of model server replicas to deploy ?

1

Model server size * ?

Small

Limits: 2 CPU, 8GiB Memory Requests: 1 CPU, 4GiB Memory

Model route

☐ Make deployed models available through an external route

Token authentication

☐ Require token authentication

Add **Cancel**

4. Click **Add**.
5. In the **Models and model servers** list, next to the new model server, click **Deploy model**.

Models and model servers ?

[Add model server](#) **Multi-model serving enabled**

Model Server Name	Serving Runtime	Deployed models	Tokens
Model Server	OpenVINO Model Server	0	Tokens disabled

Deploy model

6. In the form, specify the following values:
 - a. For **Model deployment name**, type **fraud**.
 - b. For **Model framework (name - version)**, select **onnx-1**.

- c. For **Existing connection**, select **My Storage**.
- d. Type the path that leads to the version folder that has your model file: **models/fraud**
- e. Leave the other fields with the default settings.

Deploy model ✕

Configure properties for deploying your model

Project
Fraud Detection

Model deployment name *
fraud
This is the name of the inference service created when the model is deployed
The resource name will be **fraud**.
[Edit resource name](#) ?

Model server
Model Server

Model framework (name - version) *
onnx - 1

Source model location

☒ Existing connection

? Not seeing what you're looking for?

Connection *
My Storage
[View connection details](#)

Path *
models/fraud
Enter a path to a model or folder. This path cannot point to a root folder.

☐ Create connection

Deploy **Cancel**

7. Click **Deploy**.

Verification

Notice the loading symbol under the **Status** section. The symbol changes to a checkmark when the deployment completes successfully.

API protocol

Status

REST



Next step

[Testing the model API](#)

4.3. TESTING THE MODEL API

After you deploy the model, you can test its API endpoints.

Procedure

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Models** tab.
2. Take note of the model's Inference endpoint URL. You need this information when you test the model API.

If the **Inference endpoint** field has an **Internal endpoint details** link, click the link to open a text box that shows the URL details, and then take note of the **restUrl** value.

Inference endpoint

Inference endpoints



url

<https://fraud.fraud-detection.svc.cluster.local>

[Internal endpoint details](#)

3. Return to the JupyterLab environment and try out your new endpoint.
If you deployed your model with the multi-model serving platform, follow the directions in **3_rest_requests_multi_model.ipynb** to try a REST API call and **4_grpc_requests_multi_model.ipynb** to try a gRPC API call.

If you deployed your model with the single-model serving platform, follow the directions in **5_rest_requests_single_model.ipynb** to try a REST API call.

Next step

(Optional) [Automating workflows with data science pipelines](#)

(Optional) [Running a data science pipeline generated from Python code](#)

CHAPTER 5. IMPLEMENTING PIPELINES

5.1. AUTOMATING WORKFLOWS WITH DATA SCIENCE PIPELINES

Earlier, you used a notebook to train and save your model. Optionally, you can automate these tasks by using Red Hat OpenShift AI pipelines. Pipelines offer a way to automate the execution of many notebooks and Python code. By using pipelines, you can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.

To explore the pipeline editor, complete the steps in the following procedures to create your own pipeline.

Alternately, you can skip the following procedures and instead run the **6 Train Save.pipeline** file.

5.1.1. Creating a pipeline

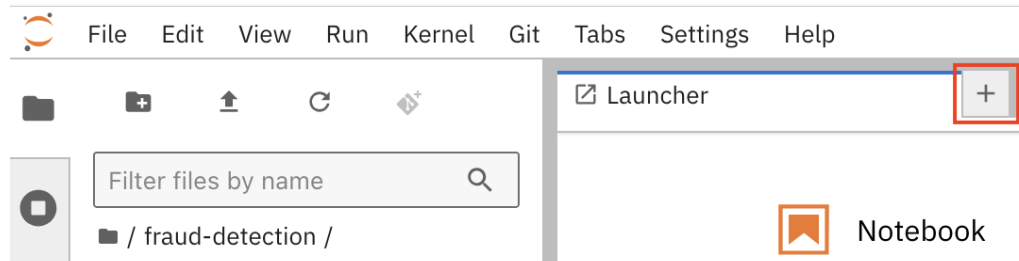
You can create a simple pipeline by using the GUI pipeline editor. The pipeline uses the notebook that you used earlier to train a model and then save it to S3 storage.

Prerequisites

- You configured a pipeline server as described in [Enabling data science pipelines](#).
- If you configured the pipeline server after you created your workbench, you stopped and then started your workbench.

Procedure

1. Open your workbench's JupyterLab environment. If the launcher is not visible, click + to open it.

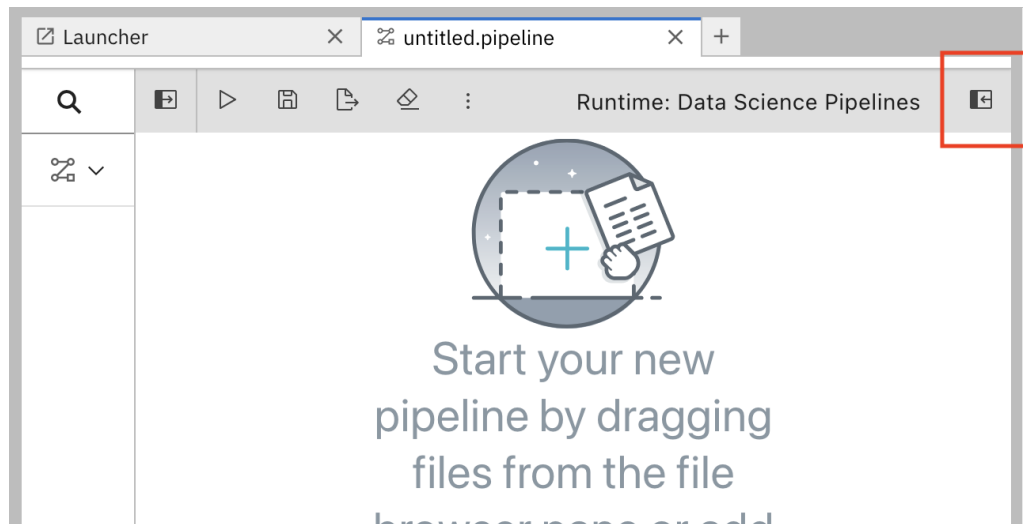


2. Click **Pipeline Editor**.

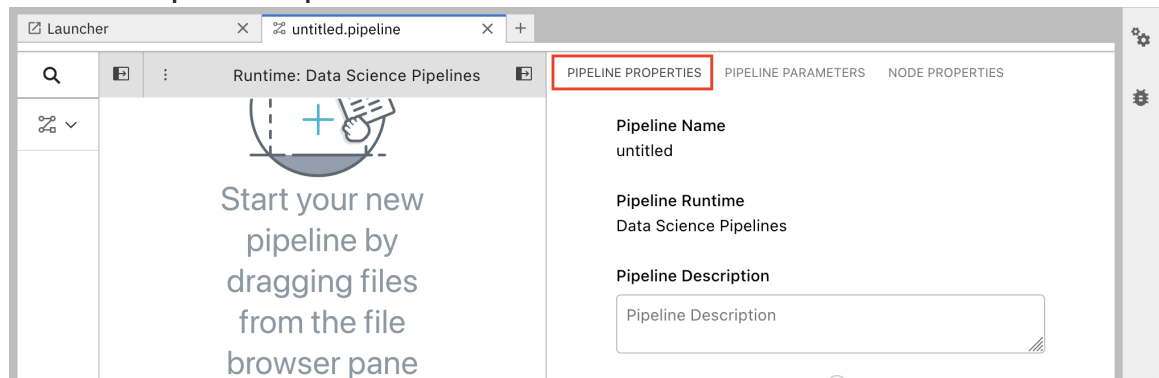


You have created a blank pipeline.

3. Set the default runtime image for when you run your notebook or Python code.
 - a. In the pipeline editor, click **Open Panel**.



- b. Select the **Pipeline Properties** tab.



- c. In the **Pipeline Properties** panel, scroll down to **Generic Node Defaults** and **Runtime Image**. Set the value to **Tensorflow with Cuda and Python 3.11 (UBI 9)**.

PIPELINE PROPERTIES PIPELINE PARAMETERS NODE PROPERTIES

Memory Size (GB)

Kubernetes Pod Labels Data Volumes Kubernetes Tolerations Kubernetes Pod Annotations Generic Node Defaults Runtime Image Kubernetes Secrets Environment Variables Custom Node Defaults

4. Select **File** → **Save Pipeline**.

Verification

- In the file-browser window, you can see the pipeline file that you created.

Next step

[Adding nodes to your pipeline](#)

5.1.2. Adding nodes to your pipeline

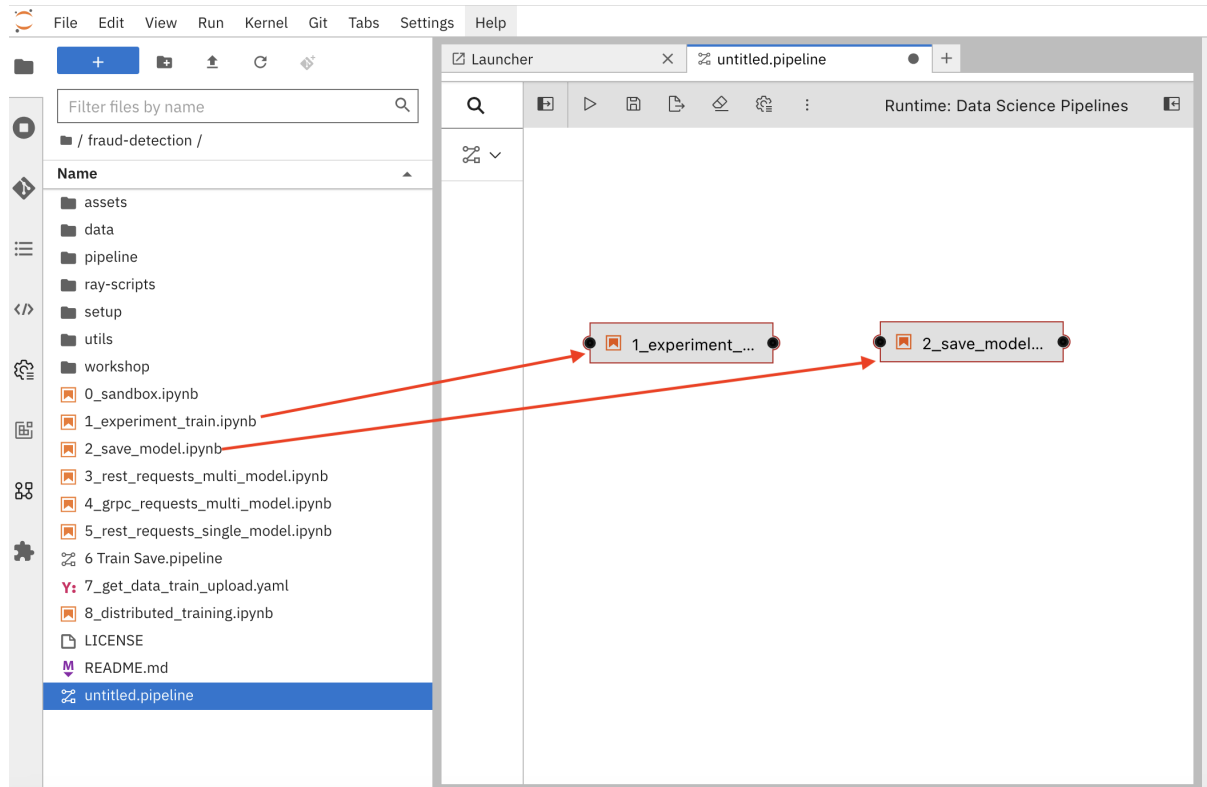
Add some steps, or **nodes**, in your pipeline for the **1_experiment_train.ipynb** and **2_save_model.ipynb** notebooks.

Prerequisites

- You created a pipeline file as described in [title](#)

Procedure

- From the JupyterLab file-browser panel, drag the **1_experiment_train.ipynb** and **2_save_model.ipynb** notebooks onto the pipeline canvas.



- Click the output port of **1_experiment_train.ipynb** and drag a connecting line to the input port of **2_save_model.ipynb**.



- Save the pipeline.

Verification

- Your pipeline has two nodes.

Next step

[Specify the training file as a dependency](#)

5.1.3. Specifying the training file as a dependency

Set node properties in your pipeline to specify the training file as a dependency.



NOTE

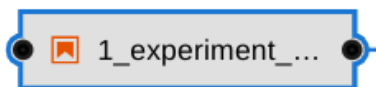
If you do not set this file dependency, the file is not included in the node when it runs and the training job fails.

Prerequisites

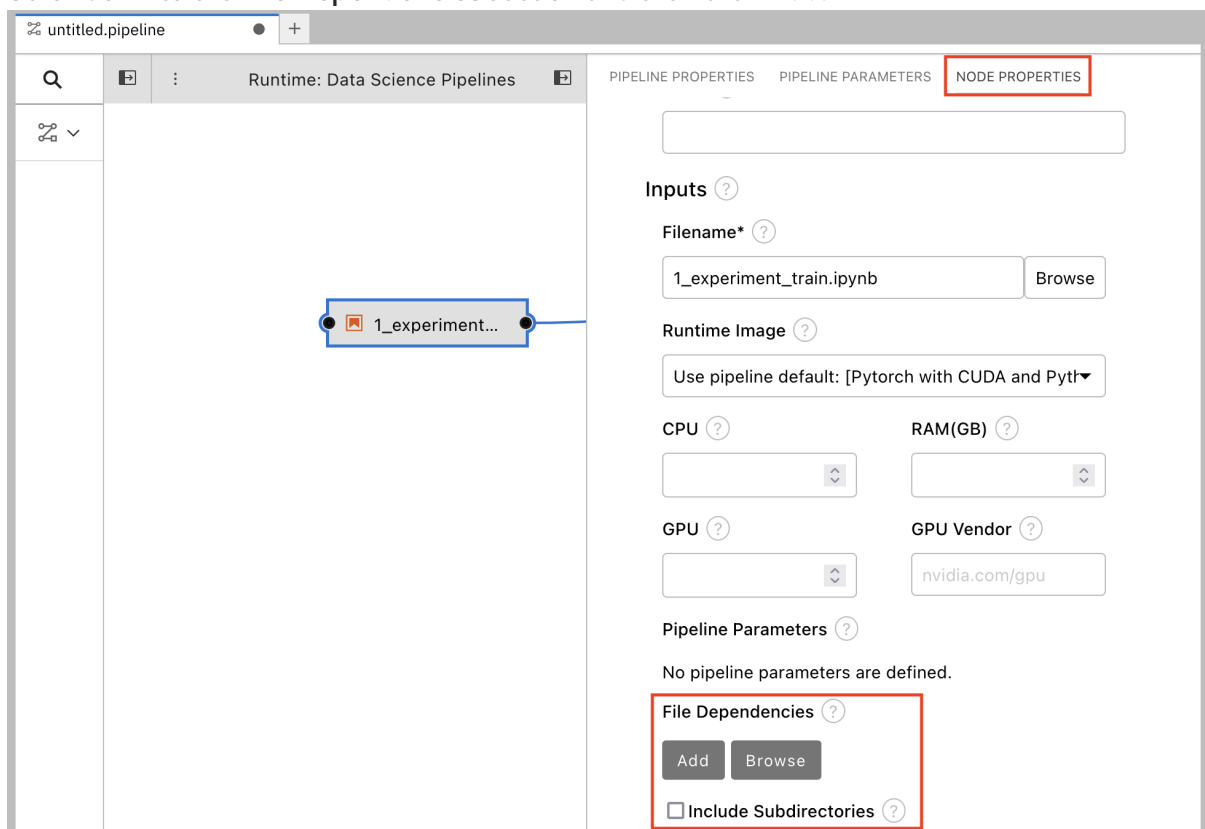
- You added two nodes to your pipeline as described in [Adding nodes to your pipeline](#).

Procedure

- Click the **1_experiment_train.ipynb** node.



- In the **Properties** panel, click the **Node Properties** tab.
- Scroll down to the **File Dependencies** section and then click **Add**.



- Set the value to **data/*.csv** which has the data to train your model.
- Select the **Include Subdirectories** option.

File Dependencies

☒ **Include Subdirectories** 

6. **Save** the pipeline.

Verification

- The training file is a dependency of the first node in your pipeline.

Next step

[Creating and storing the ONNX-formatted output file](#)

5.1.4. Creating and storing the ONNX-formatted output file

You must set the **models/fraud/1/model.onnx** file as the output file for both nodes in your pipeline.

Prerequisites

- You added the training file as a dependency to the first node in your pipeline, as described in [Specifying the training file as a dependency](#).

Procedure

1. Select node 1.
2. Select the **Node Properties** tab.
3. Scroll down to the **Output Files** section, and then click **Add**.
4. Set the value to **models/fraud/1/model.onnx**.

Outputs

Output Files

5. Repeat steps 2-4 for node 2.

6. Click **Save Pipeline**.

Verification

- In node 1, the notebook creates the **models/fraud/1/model.onnx** file.
- In node 2, the notebook uploads the **models/fraud/1/model.onnx** file to the S3 storage bucket.

Next step

[Configuring the connection to storage](#)

5.1.5. Configuring the connection to storage

In node 2, the notebook uploads the model to the S3 storage bucket. You must set the S3 storage bucket keys by using the secret created by the **My Storage** connection that you set up in [Storing data with connections](#).

You can use this secret in your pipeline nodes without having to save the information in your pipeline code. This is important, for example, if you want to save your pipelines - without any secret keys - to source control.

The name of the secret is **aws-connection-my-storage**.

NOTE

If you named your connection something other than **My Storage**, you can obtain the secret name in the OpenShift AI dashboard by hovering over the help (?) icon in the **Connections** tab.

((o))

Connections
?

Name ↑

My Storage ?

Resource names and types are used to find your resources in OpenShift. ✕

Resource name

Resource type

aws-connection-my-storage 📋

Secret

The **aws-connection-my-storage** secret includes the following fields:

- **AWS_ACCESS_KEY_ID**
- **AWS_DEFAULT_REGION**
- **AWS_S3_BUCKET**
- **AWS_S3_ENDPOINT**

- **AWS_SECRET_ACCESS_KEY**

You must set the secret name and key for each of these fields.

Prerequisites

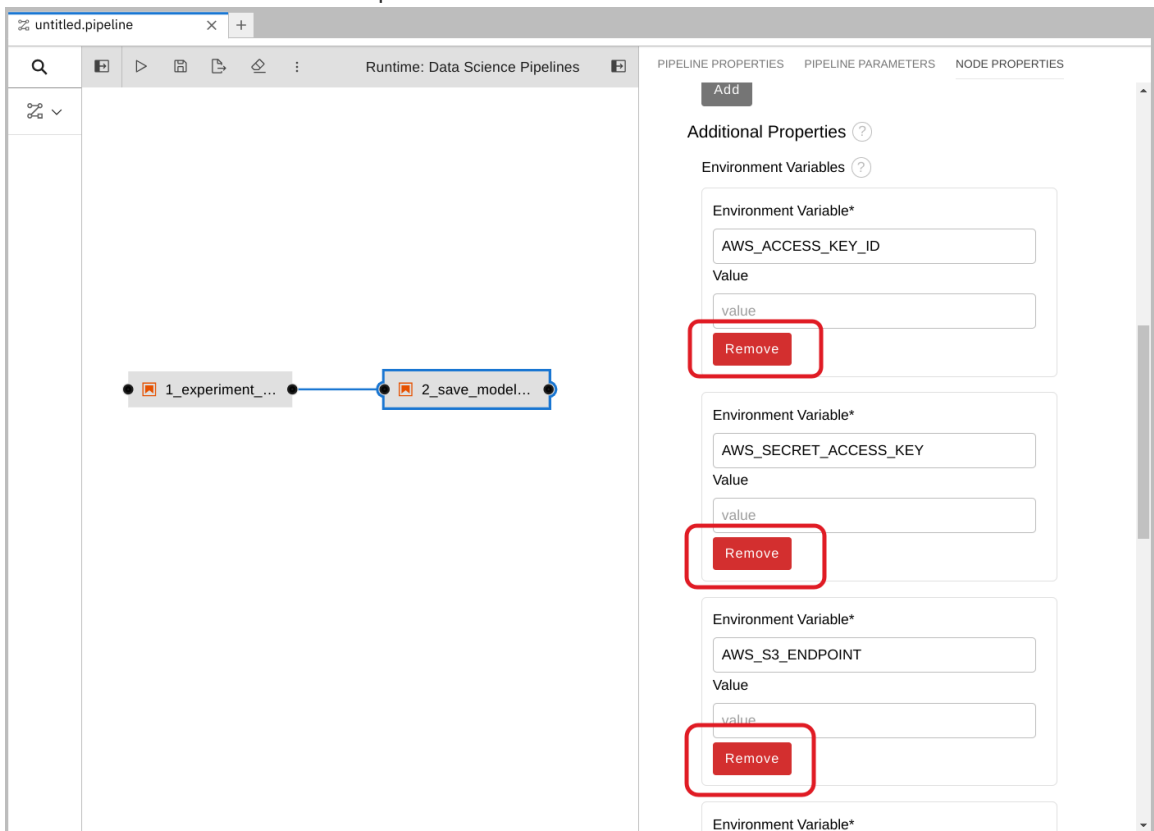
- You created the **My Storage** connection, as described in [Storing data with connections](#).
- You set the **models/fraud/1/model.onnx** file as the output file for both nodes in your pipeline, as described in [Creating and storing the ONNX-formatted output file](#).

Procedure

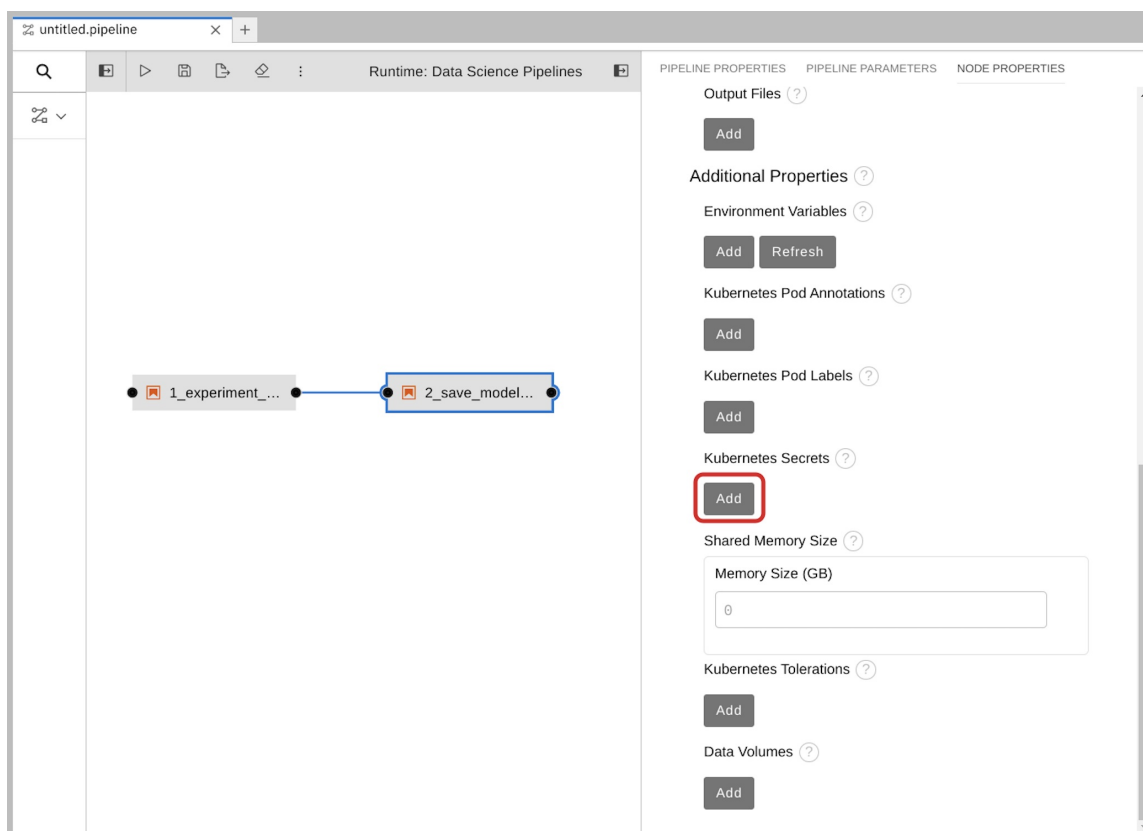
1. Remove any pre-filled environment variables.
 - a. Select node 2, and then select the **Node Properties** tab. Under **Additional Properties**, note that some environment variables have been pre-filled. The pipeline editor inferred that you need them from the notebook code.

Because you do not want to save the value in your pipelines, remove all of these environment variables.

- b. Click **Remove** for each of the pre-filled environment variables.



2. Add the S3 bucket and keys by using the Kubernetes secret.
 - a. Under **Kubernetes Secrets**, click **Add**.



b. Enter the following values and then click **Add**.

- Environment Variable: **AWS_ACCESS_KEY_ID**
 - Secret Name: **aws-connection-my-storage**
 - Secret Key: **AWS_ACCESS_KEY_ID**

Kubernetes Secrets ?

Environment Variable*

Secret Name*

Secret Key*

Remove

Add

3. Repeat Step 2 for each of the following Kubernetes secrets:

- Environment Variable: **AWS_SECRET_ACCESS_KEY**
 - Secret Name: **aws-connection-my-storage**
 - Secret Key: **AWS_SECRET_ACCESS_KEY**
- Environment Variable: **AWS_S3_ENDPOINT**
 - Secret Name: **aws-connection-my-storage**
 - Secret Key: **AWS_S3_ENDPOINT**
- Environment Variable: **AWS_DEFAULT_REGION**
 - Secret Name: **aws-connection-my-storage**
 - Secret Key: **AWS_DEFAULT_REGION**
- Environment Variable: **AWS_S3_BUCKET**
 - Secret Name: **aws-connection-my-storage**
 - Secret Key: **AWS_S3_BUCKET**

4. Select **File → Save Pipeline As** to save and rename the pipeline. For example, rename it to **My Train Save.pipeline**.

Verification

- You set the S3 storage bucket keys by using the secret created by the **My Storage** connection.

Next step

[Running your pipeline](#)

5.1.6. Running your pipeline

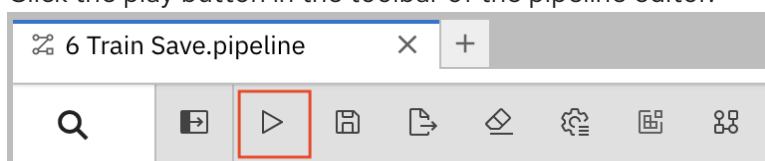
Upload the pipeline on your cluster and run it. You can do so directly from the pipeline editor. You can use your own newly created pipeline or the pipeline in the provided **6 Train Save.pipeline** file.

Prerequisites

- You set the S3 storage bucket keys, as described in [Configuring the connection to storage](#).

Procedure

1. Click the play button in the toolbar of the pipeline editor.



2. Enter a name for your pipeline.

3. Verify that the **Runtime Configuration:** is set to **Data Science Pipeline**.

4. Click **OK**.

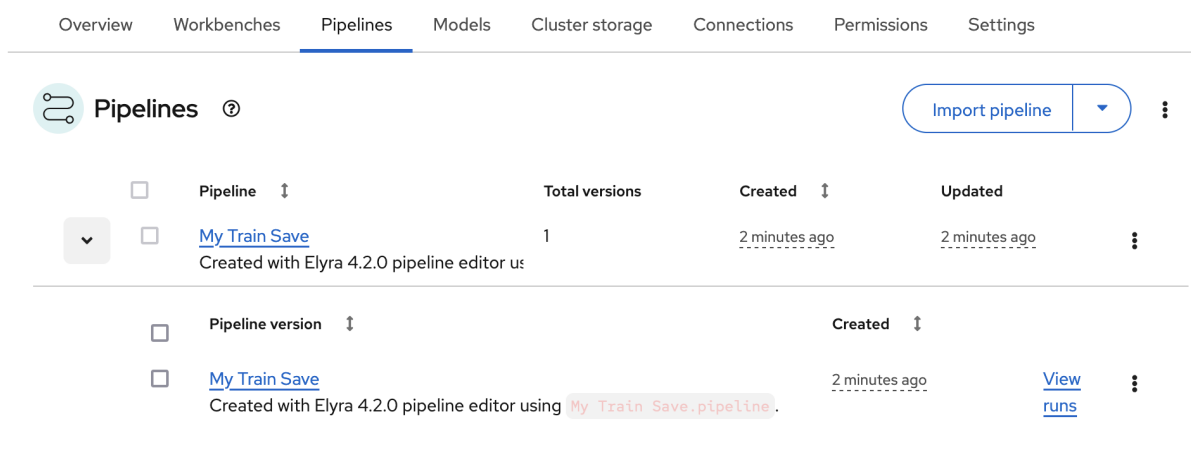
NOTE

If you see an error message stating that "no runtime configuration for Data Science Pipeline is defined", you might have created your workbench before the pipeline server was available.

To address this error, you must verify that you configured the pipeline server and then restart the workbench.

1. Follow these steps in the OpenShift AI dashboard:
2. Check the status of the pipeline server:
 - a. In your Fraud Detection project, click the **Pipelines** tab.
 - If you see the **Configure pipeline server** option, follow the steps in [Enabling data science pipelines](#).
 - If you see the **Import a pipeline** option, the pipeline server is configured. Continue to the next step.
3. Restart your Fraud Detection workbench:
 - a. Click the **Workbenches** tab.
 - b. Click **Stop** and then click **Stop workbench**.
 - c. After the workbench status is **Stopped**, click **Start**.
 - d. Wait until the workbench status is **Running**.
4. Return to your workbench's JupyterLab environment and run the pipeline.

5. In the OpenShift AI dashboard, open your data science project and expand the newly created pipeline.



The screenshot shows the OpenShift AI dashboard with the 'Pipelines' tab selected. The dashboard has a top navigation bar with tabs: Overview, Workbenches, Pipelines (active), Models, Cluster storage, Connections, Permissions, and Settings. Below the navigation bar, there's a 'Pipelines' section with a search icon and a 'Import pipeline' button. A table lists the pipelines:

	Pipeline ↓	Total versions	Created ↓	Updated	
▼	☐ My Train Save Created with Elyra 4.2.0 pipeline editor us	1	2 minutes ago	2 minutes ago	⋮

Below the main table, there's a section for 'Pipeline version' with a table:

	Pipeline version ↓	Created ↓	
▼	☐ My Train Save Created with Elyra 4.2.0 pipeline editor using <code>My Train Save.pipeline</code> .	2 minutes ago	View runs ⋮

6. Click **View runs**.

Manage your experiment runs and schedules.

Project Fraud Detection ▼

Runs Archive Schedules

Run ▼

Search...

Create run

Pipeline version My Train Save ✕

[Clear all filters](#)



Run ↑↓

Pipeline version

Experin



[My Train Save-0120013824](#)

[My Train Save](#)

[my trai](#)

One-off

7. Click your run and then view the pipeline run in progress.

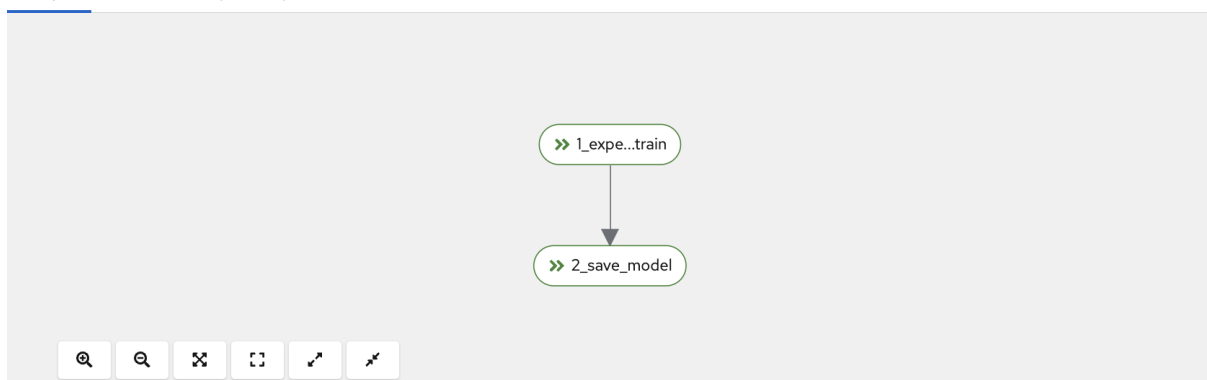
MyTrainSave1-1024180103

One-off

Running

Actions ▼

Graph Details Pipeline spec



Verification

The **models/fraud/1/model.onnx** file is in your S3 bucket. You can serve the model, as described in [Preparing a model for deployment](#).

Next step

(Optional) [Running a data science pipeline generated from Python code](#)

5.2. RUNNING A DATA SCIENCE PIPELINE GENERATED FROM PYTHON CODE

Earlier, you created a simple pipeline by using the GUI pipeline editor. You might want to create pipelines by using code that can be version-controlled and shared with others. The [Kubeflow pipelines \(kfp\)](#) SDK provides a Python API for creating pipelines. The SDK is available as a Python package that you can install by using the **pip install kfp** command. With this package, you can use Python code to create a pipeline and then compile it to YAML format. Then you can import the YAML code into OpenShift AI.

This tutorial does not describe the details of how to use the SDK. Instead, it provides the files for you to view and upload.

1. Optionally, view the provided Python code in your JupyterLab environment by navigating to the **fraud-detection-notebooks** project's **pipeline** directory. It contains the following files:
 - **7_get_data_train_upload.py** is the main pipeline code.
 - **build.sh** is a script that builds the pipeline and creates the YAML file.
For your convenience, the output of the **build.sh** script is provided in the **7_get_data_train_upload.yaml** file. The **7_get_data_train_upload.yaml** output file is located in the top-level **fraud-detection** directory.
2. Right-click the **7_get_data_train_upload.yaml** file and then click **Download**.
3. Upload the **7_get_data_train_upload.yaml** file to OpenShift AI.
 - a. In the OpenShift AI dashboard, navigate to your data science project page. Click the **Pipelines** tab and then click **Import pipeline**.
 - b. Enter values for **Pipeline name** and **Pipeline description**.
 - c. Click **Upload** and then select **7_get_data_train_upload.yaml** from your local files to upload the pipeline.

Import pipeline ✕

Fraud Detection

Pipeline name *

Python pipeline

Cannot exceed 255 characters

Pipeline description

Pipeline written in Python

Cannot exceed 255 characters

☒ Upload a file ☐ Import by url

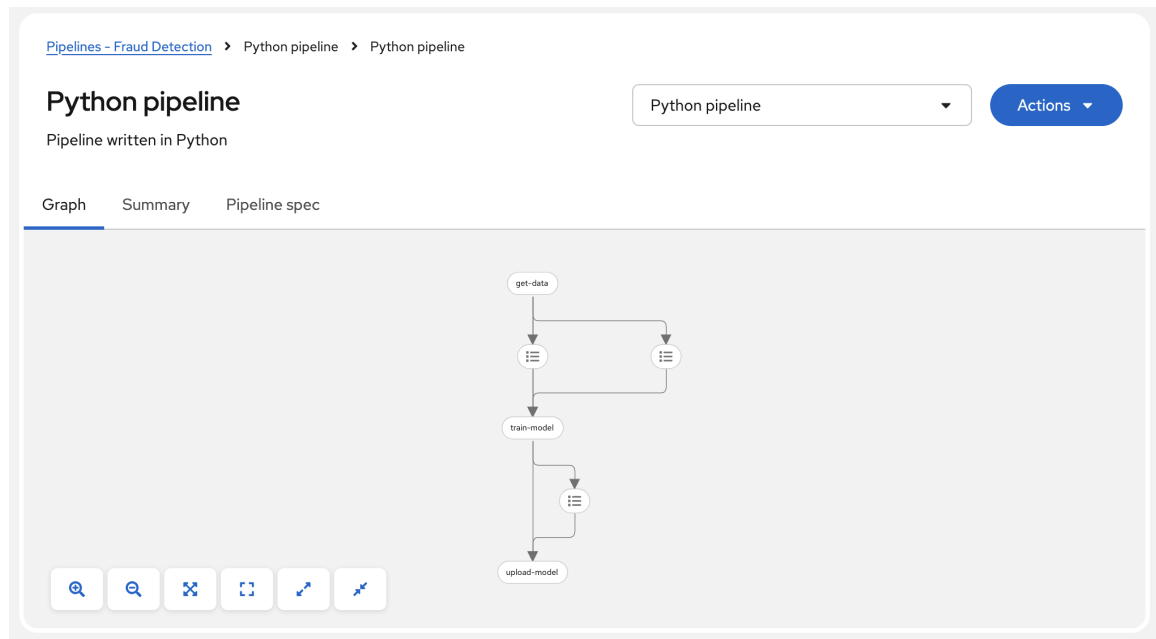
i For expected file format, refer to [Compile Pipeline Documentation](#).

[View documentation](#) [🔗](#)

7_get_data_train_upload.yaml Upload Clear

Import pipeline Cancel

- d. Click **Import pipeline** to import and save the pipeline.
The pipeline shows in graphic view.



4. Select **Actions** → **Create run**.
5. On the **Create run** page, provide the following values:
 - a. For **Experiment**, leave the value as **Default**.
 - b. For **Name**, type any name, for example **Run 1**.
 - c. For **Pipeline**, select the pipeline that you uploaded.
You can leave the other fields with their default values.

Create run

Jump to section

Run type

Project and experiment

Run details

Pipeline

Parameters

Run type

Run once immediately after creation

Project and experiment

Project

Fraud Detection

Experiment *

Default

+ Create new experiment

Run details

Name *

Run 1

Cannot exceed 255 characters

Description

Cannot exceed 255 characters

Pipeline

Pipeline *

Python pipeline

Create run

Cancel

- Click **Create run** to create the run.
A new run starts immediately.

[Runs - Fraud Detection](#) > Run 1

Run 1

One-off

⌚ Running

Graph

Details

Input parameters

Pipeline spec

CHAPTER 6. RUNNING A DISTRIBUTED WORKLOAD

You can distribute the training of a machine learning model across many CPUs by using Ray or the Training Operator.

6.1. DISTRIBUTING TRAINING JOBS

Earlier, you trained the fraud detection model directly in a notebook and then in a pipeline. You can also distribute the training of a machine learning model across many CPUs.

Distributing training is not necessary for a simple model. However, by applying it to the example fraud model, you learn how to train more complex models that require more compute power.

You can try one or both of the following options:

- The Ray distributed computing framework, as described in [Distributing training jobs with Ray](#).
- The Training Operator, as described in [Distributing training jobs with the Training Operator](#).

6.1.1. Distributing training jobs with Ray

You can use Ray, a distributed computing framework, to parallelize Python code across many CPUs or GPUs.

In your notebook environment, open the **8_distributed_training.ipynb** file and follow the instructions directly in the notebook. The instructions guide you through setting authentication, creating Ray clusters, and working with jobs.

Optionally, if you want to view the Python code for this step, you can find it in the **ray-scripts/train_tf_cpu.py** file.

The screenshot shows a Jupyter Notebook environment. On the left, a file explorer shows a directory structure for 'fraud-detection' with various subdirectories and files. The main notebook area displays the '8_distributed_training.ipynb' file. The notebook title is 'Training the Fraud Detection model with Ray by using Codeflare'. The content includes an introduction to Ray for distributed training, a code cell for installing 'codeflare-sdk' (0.19.1), and a section for 'Preparing the data' which includes code to import sys and sys.path.append to use local utilities.

For more information about TensorFlow training on Ray, see the [Ray TensorFlow guide](#).

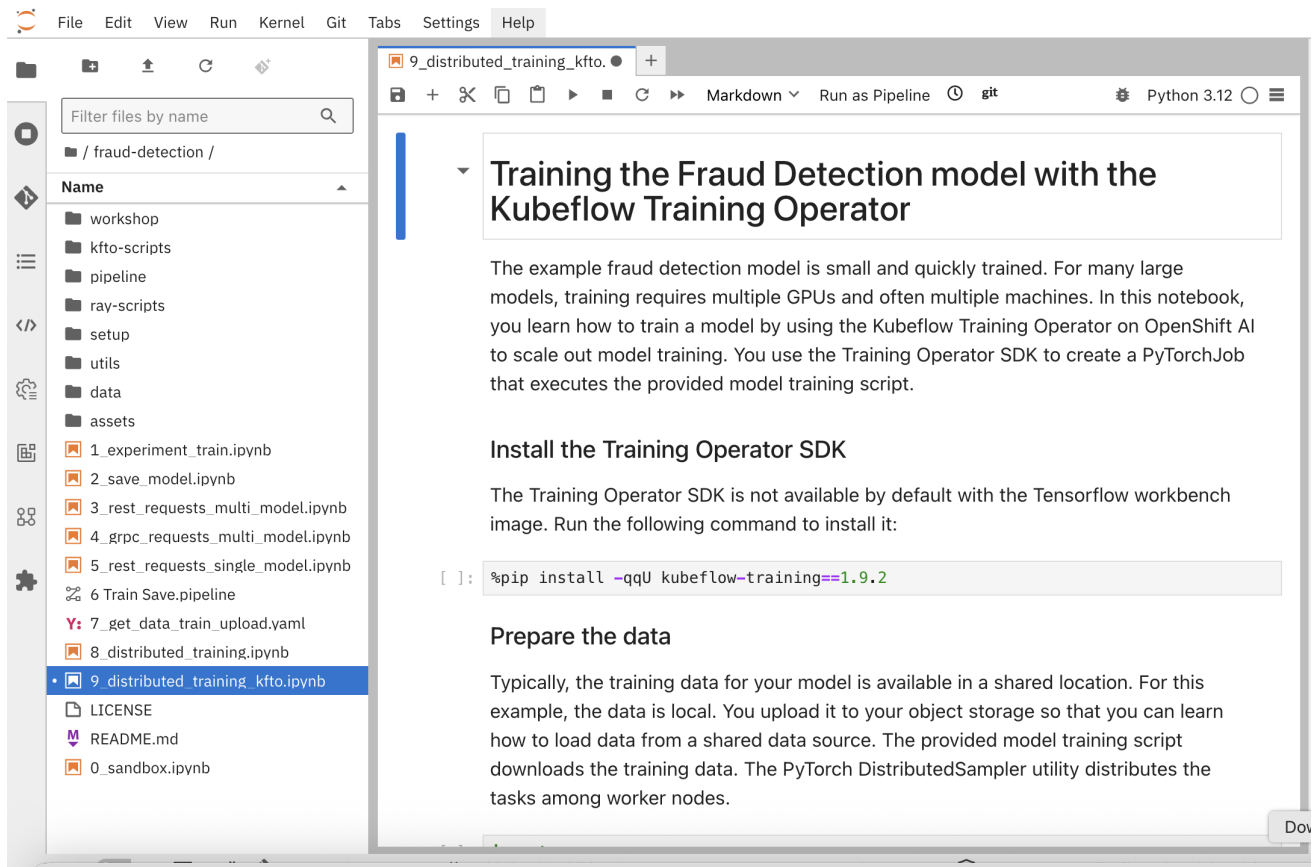
6.1.2. Distributing training jobs with the Training Operator

The Training Operator is a tool for scalable distributed training of machine learning (ML) models created with various ML frameworks, such as **PyTorch**.

You can use the Training Operator to distribute the training of a machine learning model across many hardware resources.

In your notebook environment, open the **9_distributed_training_kfto.ipynb** file and follow the instructions directly in the notebook. The instructions guide you through setting authentication, initializing the Training Operator client, and submitting a **PyTorchJob**.

Optionally, you can view the complete Python code in the **kfto-scripts/train_pytorch_cpu.py** file.



For more information about PyTorchJob training with the Training Operator, see the [Training Operator PyTorchJob guide](#).

CHAPTER 7. CONCLUSION

Congratulations. In this tutorial, you learned how to incorporate data science, artificial intelligence, and machine learning into an OpenShift development workflow.

You used an example fraud detection model and completed the following tasks:

- Explored a pre-trained fraud detection model by using a Jupyter notebook.
- Deployed the model by using OpenShift AI model serving.
- Refined and trained the model by using automated pipelines.
- Learned how to train the model by using distributed computing frameworks.